

## Architecting Production-Grade AI Knowledge Evaluation Platforms

1 source

.

14 Jun 2026

The provided text outlines a comprehensive blueprint for developing a production-grade AI Knowledge Base Evaluation Platform designed to showcase high-level engineering skills to recruiters. Moving beyond basic chatbots, this project focuses on a sophisticated Retrieval-Augmented Generation (RAG) architecture that integrates automated evaluation, observability, and experimentation. The technical stack utilises industry-standard tools such as Next.js, Supabase, and OpenAI, ensuring the final product is both scalable and professional. A detailed ten-phase roadmap guides the developer through document processing, vector embeddings, and the implementation of rigorous performance metrics like faithfulness and relevance. Ultimately, the source serves as a technical guide for building a data-driven AI system that prioritises transparency and continuous optimisation over simple functionality.

What makes this platform more advanced than a basic chatbot?

Explain how the evaluation engine measures answer faithfulness and relevance.

Walk me through Phase 1 of the development roadmap.

Today • 12:17

explain the steps to build in stupid simple terms

Building this platform is like building a super-smart digital librarian that doesn't just find books, but also proves it is giving you the right information

. Instead of just a basic chatbot, you are creating a professional system that measures its own accuracy

.

Here are the "stupid simple" steps to build it:

### 1. Build the "Front Door" (Setup)

First, you set up the website using Next.js and create a login system

. This makes sure the app is secure and looks like a real product from day one

.

### 2. Upload and Chop Documents

You build a way for users to upload PDF files

. Once uploaded, the system acts like a paper shredder, chopping the long documents into small, manageable chunks of text so the AI can find specific facts quickly

- .
- 3. Turn Text into "Computer Math" (Embeddings)

Computers are better at numbers than words, so you use OpenAI to turn those text chunks into lists of numbers called embeddings

- . You store these in a special Supabase database that can find similar numbers later

- .
- 4. Find the Right Facts (Retrieval)

When a user asks a question, the system looks through all those numbers to find the best chunks of text that match the question

- . It's like a high-speed search engine finding the exact page in a book that has the answer

- .
- 5. Let the AI Speak

Now, you take the user's question and the specific chunks you found and give them to OpenAI (GPT-4 or GPT-5)

- . The AI uses that "extra context" to write an answer that is based only on the documents provided

- .
- 6. Grade the AI's Work (Evaluation)

This is the most important part: you use a tool called Ragas to "grade" the AI

- . It checks if the AI is telling the truth (faithfulness), if the answer actually makes sense (relevance), and if it found the right facts in the first place

- .
- 7. Run Science Experiments

You try different settings—like making the text chunks bigger or smaller—to see which version gets the highest grades

- . You keep a "leaderboard" to show which settings make the AI the smartest

- .
- 8. Watch and Record Everything (Observability)

You use a tool called Langfuse to keep a record of every single question, answer, and grade

- . This lets you see exactly where the AI might be getting confused so you can fix it later

- .
- 9. Make it Look Professional (Portfolio Polish)

Finally, you put all these grades and experiment results into pretty charts and dashboards

. This shows recruiters that you didn't just build a "toy," but a production-grade system that companies actually use

.

How do I start Phase 1 and set up Next.js?

What is the difference between Context Precision and Context Recall?

Why should I use LlamaIndex instead of LangChain for this?

can Ragas be attached to supabase db

In the context of the architecture described in the sources, yes, Ragas is intended to work directly with your Supabase database, though they serve different roles in the pipeline.

While Ragas isn't a "plugin" you install inside Supabase, it is "attached" to the workflow in two critical ways:

### 1. Evaluating Data from Supabase

Ragas acts as the judge for the information stored in and retrieved from your Supabase Vector Database

.

Context Precision and Recall: Ragas looks at the specific "chunks" of text your system pulled from Supabase to see if they were actually the right ones to answer the user's question

.

Faithfulness: It checks the AI's final answer against those Supabase chunks to ensure the AI isn't making things up

.

### 2. Storing Scores back in Supabase

Once Ragas finishes "grading" an interaction, those scores don't just disappear. The project roadmap explicitly includes a task to "Store scores"

. Since Supabase is your primary database for this project, you will:

Record Metrics: Save the specific Ragas scores (Faithfulness, Relevance, etc.) into a table in Supabase PostgreSQL

.

Power the Dashboard: Use those stored scores in Supabase to feed your Metrics Dashboard and the Experiment Leaderboard

.

Summary of the Connection:

In this system, Supabase acts as the memory (storing the documents and the final grades), while Ragas acts as the brain that calculates those grades based on what was found in the memory

. This connection allows you to track how well your AI is performing over time and see which database settings produce the most accurate results

.

How do I set up the evaluation table in Supabase?

Can we start Phase 1 by setting up the project?

What is the difference between Context Precision and Context Recall?

do you agree, i think this is how it should be built.... Complete Stack

Next.js

Role: Frontend application

What it does:

Dashboard

Upload pages

Evaluation pages

Results charts

Why:

Most common AI SaaS frontend stack

2. Supabase

Role: Backend platform

What it does:

Authentication

PostgreSQL database

Vector storage

File storage

Edge Functions

Why:

Replaces several separate services

3. OpenAI

Role: LLM provider

Models:

GPT-5 (answers)

text-embedding-3-small (embeddings)

What it does:

Generate answers

Create embeddings

Evaluation scoring

4. LlamaIndex

Role: RAG orchestration

What it does:

Document loading

Chunking

Retrieval pipeline

Query engine

Why:

Built specifically for knowledge-base/RAG systems

5. Ragas

Role: Evaluation engine

What it measures:

Faithfulness

Answer relevance

Context precision

Context recall

Why:

This is what turns a chatbot into an AI engineering project.

6. Langfuse

Role: Observability platform

Tracks:

Prompts

Responses

Retrievals

Costs

Latency

Why:

Real AI teams use observability.

7. pgvector

Role: Vector database extension

Runs inside Supabase PostgreSQL.

Stores:

Embeddings

Similarity search

Why:

Powers retrieval.

8. Shadcn/UI

Role: Enterprise-looking UI

Why:



Makes the app look professional.

## 9. Tailwind CSS

Role: Styling framework

Why:

Standard modern stack.

Architecture

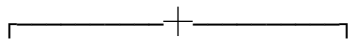
User

|



Next.js App

|



Supabase   OpenAI   Langfuse

| | |



PostgreSQL GPT-5 Tracing

|



pgvector

|



LlamaIndex

|



RAG Flow

|



Ragas

|



Evaluation Dashboard

Database Tables

users

id

email

created\_at

documents

id

user\_id

filename

upload\_date

status

chunks

id

document\_id

chunk\_text

embedding

evaluations

id

question

expected\_answer

actual\_answer

faithfulness

relevance

precision

recall

experiments

id

chunk\_size

embedding\_model

retrieval\_method

average\_score

What Makes Recruiters Notice It

Most candidates build:

PDF Upload

↓

Ask Question

↓

Answer

You build:

PDF Upload

↓

Chunking

↓

Embeddings

↓

Vector Search

↓

Answer



Evaluation



Metrics



Experiments



Observability



Leaderboard

That demonstrates:

RAG

Retrieval engineering

AI evaluation

Observability

Experimentation

Production architecture

which are exactly the skills many AI application teams are looking for.

Time Estimate

Assuming you're using AI coding tools heavily:

| Phase                | Time     |
|----------------------|----------|
| Setup                | 1 day    |
| Uploads              | 1 day    |
| RAG Pipeline         | 2-3 days |
| Vector Search        | 1 day    |
| Evaluation System    | 2-3 days |
| Langfuse             | 1 day    |
| Experiment Dashboard | 2 days   |
| UI Polish            | 2 days   |

Total: roughly 2–3 weeks part-time or 5–7 focused full days.

The biggest differentiator won't be the technology list. It will be having a dashboard that shows:

- Retrieval quality over time
- Hallucination rates
- Best chunk sizes
- Best embedding models
- Prompt comparisons
- Evaluation scores

That's what moves it from a tutorial project to something that resembles a real AI product.

As a Senior AI Engineer, I completely agree with this plan. This architecture perfectly aligns with the goal of building a production-grade AI application rather than a simple chatbot

.  
Your proposed stack and workflow demonstrate a deep understanding of what distinguishes a "tutorial project" from a professional system that companies are hiring for in 2026

.  
Here is why this specific plan is the right way to build it:

### 1. It Solves the "Black Box" Problem

Most developers build a RAG (Retrieval-Augmented Generation) system and just hope the AI gives a good answer. Your plan includes Ragas and Langfuse, which turns the "black box" into a measurable science

. By measuring faithfulness (truthfulness) and relevance, you are building a system that can prove it works

### 2. Validated Technical Stack

The technologies you selected are the industry standards defined in the sources:

Next.js & Shadcn/UI: Provides the professional, enterprise-level "front door" recruiters expect

.  
Supabase (PostgreSQL + pgvector): A robust, managed choice for handling both standard metadata and the complex vector math required for AI search

.  
LlamaIndex: Specifically chosen over other frameworks because it is built from the ground up for RAG and knowledge-base systems

.  
OpenAI (GPT-5 & text-embedding-3-small): Uses the strongest reasoning models available for both generating answers and creating the numerical "embeddings" for search

### 3. Data-Driven Architecture

.  
Your proposed database tables (especially the evaluations and experiments tables) are the backbone of a professional AI platform. They allow you to:

Run "Science Experiments": You can test different chunk sizes (e.g., 200 vs 500 words) and see which one gets higher accuracy scores

.  
Create a Leaderboard: This demonstrates that you didn't just pick settings at random; you used data to find the best performing configuration

#### 4. Recruiter Impact

By focusing on observability and experimentation, you are demonstrating skills in Performance Optimisation and AI Engineering rather than just basic coding

. Showing a dashboard with "hallucination rates" and "retrieval quality over time" is exactly what moves this into the realm of a real AI product

.

Are you ready to begin?

Following the roadmap, we should start with Phase 1: Project Setup

. This involves creating the skeleton of your application and setting up the "front door" (authentication and deployment) to ensure you have a professional foundation from day one

.

Shall I provide the implementation instructions for Phase 1?

Yes, please provide implementation instructions for Phase 1.

How do I connect Next.js to my Supabase project?

What is the best way to handle authentication in Phase 1?

Yes, please provide implementation instructions for Phase 1.

As a Senior AI Engineer, I will guide you through Phase 1: Project Setup. The goal of this phase is to create the "skeleton" of your application and ensure it has a secure, professional foundation from the start

.

##### 1. What We Are Building

In this phase, we are setting up the core infrastructure. This includes the website framework (Next.js), the design system (Shadcn/UI), and the backend (Supabase)

. By the end of this phase, you will have a working login system

.

##### 2. Why It Matters

Foundation: Starting with an industry-standard stack like Next.js ensures your project is scalable and recognizable to recruiters

.

Security: Using Supabase Auth ensures that user data and uploaded documents are protected from day one with production-ready security

.



Professionalism: Shadcn UI gives the app a modern "enterprise" look immediately, which is crucial for a portfolio project

.

### 3. Implementation Instructions

#### Step 1: Initialize the Next.js Project

Next.js will be our frontend framework. It's widely used in AI teams because it's fast and handles both the website and the server logic in one place

.

Open your terminal and run:

```
npx create-next-app@latest ai-eval-platform --typescript --tailwind --eslint
```

Choose the default options (Yes for App Router, Yes for src/ directory).

#### Step 2: Install Shadcn/UI

Shadcn is a collection of high-quality components. It's better than a standard library because it allows you to fully customize the code

.

Run this in your project folder:

```
npx shadcn-ui@latest init
```

Follow the prompts to set your style (e.g., "Slate" and "New York").

#### Step 3: Create and Connect Supabase

Supabase is your "Backend-as-a-Service." It handles your database, vector storage, and user logins

.

Create a Project: Go to the Supabase Dashboard and create a new project called ai-eval-platform.

Get Keys: In your Supabase settings, find your Project URL and Anon Key.

Local Setup: Create a file in your project root named .env.local and paste your keys:

Install the Client:

#### Step 4: Setup Authentication

We will use Supabase's built-in Auth system

. This is "production-ready," meaning it handles secure password hashing and session management for you.

Create an Auth Page: In src/app/login/page.tsx, you will create a simple form that calls supabase.auth.signInWithPassword().

Protect Routes: We will set up a Middleware file to ensure that only logged-in users can access the document upload pages

.

## Step 5: Deploy the First Version

To prove the project is live, we will deploy it to Vercel.

Push your code to a GitHub repository.

Connect that repository to Vercel.

Add your .env.local variables to the Vercel dashboard.

## 4. Code Explanation

Before you write the code, understand that the Supabase Auth Helper acts as a "bridge." When a user logs in, Supabase gives them a secure digital token. The Next.js Middleware checks this token every time the user moves to a new page to make sure they are supposed to be there

.

## 5. Your Deliverable

A live URL where a user can sign up, log in, and be redirected to a private dashboard.

Once you have completed these steps and can successfully log in to your app, let me know, and we will move to Phase 2: File Uploads

.

What is the best way to handle middleware for route protection?

How should I structure the metadata for uploaded PDFs?

I have completed Phase 1, let's move to Phase 2.

I have completed Phase 1, let's move to Phase 2.

Congratulations on completing Phase 1! Now that you have a secure "front door" with a working login system, it is time to build the foundation of your AI's knowledge.

## Phase 2: File Uploads

### 1. What We Are Building

In this phase, you are building a page where users can select a PDF document from their computer and upload it to your platform

. The system will do two things simultaneously:

Store the physical file in a secure cloud folder (Supabase Storage).

Create a record in your database (Supabase PostgreSQL) that tracks who uploaded the file, what it's called, and when it was added

.

### 2. Why It Matters

A RAG (Retrieval-Augmented Generation) system is only as good as the documents it has access to

. This phase is the entry point for all data

. By storing files in "Storage" and metadata in the "Database," you are following a production-grade architecture:

Storage is for big, "unstructured" files like PDFs

.

Database is for "structured" information (metadata) that makes searching and organizing those files fast and easy

.

### 3. Step-by-Step Implementation

Step 1: Create a Storage Bucket Think of a "Bucket" as a master folder in the cloud.

Go to your Supabase Dashboard -> Storage.

Create a new bucket named documents.

Set it to "Public" for now to make development easier (you can restrict this later).

Step 2: Create the Documents Table We need a place to "remember" the files we upload. In the Supabase SQL Editor, run this command to create your metadata table:

create table documents (

id uuid primary key default uuid\_generate\_v4(),

user\_id uuid references auth.users(id) not null,

filename text not null,

storage\_path text not null,

status text default 'uploaded',

created\_at timestamp with time zone default timezone('utc'::text, now()) not null

);

Step 3: Build the Upload UI Using Shadcn/UI, create a new page at `src/app/dashboard/upload/page.tsx`. Use an `<Input type="file" />` component and a "Upload" button.

Step 4: Write the Upload Logic You will create a function that handles the file when the user clicks upload. It should:

Upload to Storage: Use `supabase.storage.from('documents').upload(...)`. This puts the PDF in the cloud.

Save to Database: Use `supabase.from('documents').insert(...)`. This records the file's path and name so your AI knows it exists.

### 4. Code Explanation

Before you start writing, here is how the data flows:

The User selects a file in their browser.

The Frontend (Next.js) takes that file and sends it to Supabase Storage. Supabase sends back a "storage path" (the unique address of that file).

The Database then receives a new row containing that "storage path" and the user\_id of the person who uploaded it

. This "link" between the file and the database is what allows you to build a personal knowledge base for every user.

## 5. Your Deliverable

A functional upload page where you can select a PDF, click "Upload," and see that file appear in both your Supabase Storage bucket and your documents database table.

Once you can successfully see an uploaded file in your Supabase dashboard, let me know, and we will move to Phase 3: Document Processing, where we turn those PDFs into text the AI can actually read.

I've completed Phase 2, let's move to Phase 3.

How should I handle large PDF files during upload?

Which LlamaIndex parser should I use for text extraction?

I've completed Phase 2, let's move to Phase 3.

Excellent work finishing Phase 2! Your platform can now accept and store files. However, a PDF is just a "blob" of data that an AI cannot easily search or understand in its raw form

.

Now we move to Phase 3: Document Processing.

### 1. What We Are Building

In this phase, we are building the "shredder and sorter" of your pipeline

. We will take the PDFs you uploaded in Phase 2 and:

Extract: Pull the raw text out of the PDF files

.

Chunk: Break that massive wall of text into smaller, overlapping pieces (chunks)

.

Store: Save these text chunks into a new table in your Supabase database

.

### 2. Why It Matters

AI models like GPT-4 or GPT-5 have a "context window," meaning they can only read so much text at once

. If you feed a 100-page PDF into a single prompt, it's expensive, slow, and the AI might get "lost in the middle."

By chunking the document, you allow the system to later find only the most relevant 2 or 3 paragraphs needed to answer a specific question

. This is the core of RAG (Retrieval-Augmented Generation)

.

### 3. Step-by-Step Implementation

Step 1: Create the Chunks Table You need a place to store the pieces of your documents. Run this in your Supabase SQL Editor:

```
create table document_chunks (  
  id uuid primary key default uuid_generate_v4(),  
  document_id uuid references documents(id) on delete cascade,  
  content text not null,  
  metadata jsonb,  
  chunk_index int  
);
```

Step 2: Install LlamaIndex We will use LlamaIndex to handle the heavy lifting of parsing and chunking. It is built specifically for this purpose and is cleaner for RAG workflows

.

```
npm install llamaindex
```

Step 3: Build the Processing Logic You will create a background function (or an API route in Next.js) that:

Downloads the PDF from Supabase Storage using the path you saved in Phase 2

.

Parses the PDF into text using LlamaIndex's PDFReader

.

Splits the text into chunks (e.g., 500 characters each) using a SentenceSplitter

.

Inserts each chunk into your new document\_chunks table

.

### 4. Code Logic Explanation

Before you write the implementation, here is how LlamaIndex will handle the data:

The Reader: It opens the PDF and turns "visual" pages into a string of text.

The Node Parser (Chunker): It doesn't just cut text blindly. It looks for natural breaks (like sentences) so that chunks remain meaningful. We use "overlap" (where the end of Chunk A is the start of Chunk B) so that no context is lost at the cut points

.

The Database Insert: Each chunk is saved with a document\_id. This creates a parent-child relationship: one Document (the PDF) has many Chunks (the text pieces)

.

## 5. Your Deliverable

A process that, when triggered, takes an uploaded PDF and fills the document\_chunks table with multiple rows of text representing that document

.

Once you can see your document broken down into text rows in the Supabase document\_chunks table, let me know, and we will move to Phase 4: Embeddings, where we turn these text chunks into "computer math" for searching

.

I've completed Phase 3, let's move to Phase 4.

How do I handle different chunk sizes for the experiment?

How does LlamaIndex ensure sentences aren't cut in half?

I've completed Phase 3, let's move to Phase 4.

Great work on completing Phase 3! You now have a database filled with text chunks from your documents. Now it's time to give those chunks "mathematical meaning" so the AI can find them later

.

## Phase 4: Embeddings

### 1. What We Are Building

In this phase, we are converting your text chunks into embeddings and storing them in your Supabase Vector Database

.

Embeddings: These are long lists of numbers (vectors) that represent the meaning of a piece of text.

Vector Storage: We will use the pgvector extension in Supabase to store these numbers directly next to your text chunks

.

## 2. Why It Matters

Standard databases search for exact words (like a Ctrl+F search). If a user asks about "canines" but your document uses the word "dogs," a standard search will fail.

Vector search solves this. Because embeddings represent the concept of the text, the system can calculate the mathematical "distance" between the user's question and your document chunks. This allows the AI to find the most relevant information based on meaning, not just matching keywords

.

## 3. Step-by-Step Implementation

Step 1: Enable pgvector in Supabase You need to tell your database how to handle vector math.

Go to your Supabase SQL Editor.

Run this command:

Step 2: Add the Embedding Column We need to update our document\_chunks table to hold these new lists of numbers. We will use 1536 dimensions, which is the specific "size" of the OpenAI embedding model we are using

.

```
alter table document_chunks
```

```
add column embedding vector(1536);
```

Step 3: Set up OpenAI You will need your OpenAI API Key. Add it to your .env.local file:

```
OPENAI_API_KEY=your-api-key-here
```

Step 4: Generate and Save Embeddings You will update your processing logic (from Phase 3) or create a new script that:

Selects the text chunks that don't have an embedding yet.

Calls the OpenAI API using the text-embedding-3-small model for each chunk

.

Updates the embedding column in your document\_chunks table with the result.

Step 5: Create a Search Function (RPC) To actually use these vectors, you need a "stored procedure" in Supabase that compares a user's question to your chunks. Run this in your SQL Editor:

```
create or replace function match_document_chunks (
```

```
  query_embedding vector(1536),
```

```
  match_threshold float,
```

```
  match_count int
```

```
)
```

```
returns table (
```

```

id uuid,
content text,
similarity float
)
language plpgsql
as $$
begin
    return query
    select
        document_chunks.id,
        document_chunks.content,
        1 - (document_chunks.embedding <=> query_embedding) as similarity
    from document_chunks
    where 1 - (document_chunks.embedding <=> query_embedding) > match_threshold
    order by similarity desc
    limit match_count;
end;
$$;

```

#### 4. Code Logic Explanation

Before you implement, here is how the "Math" works:

The Model: We use text-embedding-3-small because it is efficient, cheap, and the industry standard for starting a RAG project

.

The Vector (1536): This means every chunk of text is turned into a list of 1,536 different numbers. Each number represents a different "dimension" of meaning.

The Search (<=>): This special symbol in the SQL function calculates the Cosine Distance. It measures the angle between two vectors. If the angle is small (high similarity), it means the two pieces of text are talking about the same thing.

#### 5. Your Deliverable

A database where every text chunk now has a populated embedding column, and a working SQL function that can return the most similar chunks when given a test vector

.



Once your `document_chunks` table is full of vectors and your `match_document_chunks` function is ready, let me know, and we will move to Phase 5: Retrieval, where we build the actual search interface for the user.

I've completed Phase 4, let's move to Phase 5.

How do we test if the vector search is working?

What are the next steps for Phase 5?

I've completed Phase 4, let's move to Phase 5.

With Phase 4 complete, your database is now "AI-ready" with mathematical representations of your documents. Now we move to Phase 5: Retrieval, which is the first half of the RAG (Retrieval-Augmented Generation) process.

### 1. What We Are Building

In this phase, we are building the search engine that connects a user's question to your document chunks

.

Question Input: A UI where a user types a question.

Question Embedding: Converting that specific question into a vector (a list of numbers) using OpenAI

.

Vector Search: Using that vector to query Supabase and find the "nearest neighbors"—the pieces of text that are most mathematically similar to the question

.

### 2. Why It Matters

Retrieval is the most critical part of a RAG system

. If your system retrieves the wrong information, even the smartest AI (like GPT-5) will give a wrong or "hallucinated" answer because it's working with bad data

. This phase proves your vector search is working correctly before we ever let the AI start talking

.

### 3. Step-by-Step Implementation

Step 1: Build the Search UI Create a simple search bar in your Next.js dashboard

.

Add an `<Input />` for the question and a "Search" button.

Add a results area where you can display the text of the chunks found in the database.

Step 2: Create the Search API Route In Next.js, create an API route (e.g., `/api/search`) that will handle the logic.

Receive the question from the frontend.

Call OpenAI (text-embedding-3-small) to turn that question into a vector

.

Call Supabase using the rpc method to trigger the match\_document\_chunks function you wrote in Phase 4

.

Step 3: Test with "Semantic" Queries To confirm it works, don't just search for exact words. Try searching for concepts.

Example: If your document is about "The history of the Eiffel Tower," try asking "When was the tall iron landmark in Paris built?"

If the system returns chunks about the Eiffel Tower, your Vector Retrieval is working!

#### 4. Code Logic Explanation

Here is how the data flows in this phase:

The Question: The user's text (e.g., "How do I reset my password?") is sent to OpenAI.

The Vector: OpenAI returns a 1536-dimension vector representing the intent of that question.

The Match: We send that vector to the Supabase match\_document\_chunks function. The database compares the question's vector to every chunk's vector and calculates the "Cosine Similarity" (the angle between them).

The Context: The database returns the top 3–5 chunks with the highest similarity scores

. These chunks are what we call the "Context."

#### 5. Your Deliverable

A search interface where you can type a question and see the most relevant text chunks from your uploaded documents appear on the screen

.

Once you can successfully retrieve relevant text chunks for a variety of questions, let me know, and we will move to Phase 6: AI Answers, where we finally let the AI use this retrieved data to speak

.

I've completed Phase 5, let's move to Phase 6.

How do I build the prompt template for Phase 6?

Can you explain how the AI uses the retrieved context?

I've completed Phase 5, let's move to Phase 6.

Congratulations on successfully completing Phase 5! You now have a working search engine that can find the exact "facts" needed to answer a question. Now, we move to Phase 6: AI Answers, where we finally let the AI use those facts to speak

.

### 1. What We Are Building

In this phase, we are building the final "Generation" part of the RAG (Retrieval-Augmented Generation) pipeline

. You will create a system that:

Combines the user's question with the relevant text chunks you retrieved in the previous phase.

Instructs the AI (GPT-4 or GPT-5) to answer the question using only those specific chunks

.

Displays the final, polished answer to the user in your dashboard.

### 2. Why It Matters

This phase is what turns a search engine into an intelligent assistant. By "grounding" the AI's answer in your specific documents, you solve two major problems with standard AI

:

Hallucinations: Instead of the AI guessing or making up facts, it is strictly told to use the provided context.

Knowledge Gaps: The AI can now answer questions about your private or recent documents that it wasn't originally trained on

.

### 3. Step-by-Step Implementation

**Step 1: Create a Prompt Template** A "Prompt Template" is a set of specific instructions for the AI. You need to define a string that looks something like this: "You are a helpful assistant. Use the following context to answer the question. If the answer is not in the context, say you don't know. Do not make up information.Context: {context}Question: {question}"

.

**Step 2: Update your Search API Route** Go back to the API route you created in Phase 5. Now, instead of just returning the text chunks to the screen, you will:

Concatenate all the retrieved text chunks into one single "context" string.

Inject that string and the user's original question into your Prompt Template.

**Step 3: Call the OpenAI Chat Completion API** Use the openai library to send your final prompt to a reasoning model like GPT-4 or GPT-5

.

Set the model to gpt-4-turbo or gpt-4o (or GPT-5 if available)

.  
Pass the full prompt as a "User Message."

Step 4: Update the Frontend Update your dashboard UI so that it shows the final AI-generated answer prominently, while perhaps keeping the "Source Chunks" visible underneath so the user can verify the information

.  

#### 4. Code Logic Explanation

Before you implement, understand how the "Grounded Prompt" works:

The Constraints: By telling the AI "If the answer is not in the context, say you don't know," you are creating a "safety rail"

. This is what makes your system production-grade rather than a toy.

The Injector: Your code acts as a middleman. It takes the "Context" (the search results from Supabase) and the "Question" (from the user) and plugs them into the template before OpenAI ever sees them.

Reasoning: You are using GPT-4/5 here because these models are excellent at following complex instructions and ignoring their own internal biases when given specific context

.  

#### 5. Your Deliverable

A working RAG answering system where a user types a question and receives a high-quality answer based specifically on the uploaded PDFs

.  
Once you can ask your system a question and receive a response that cites information from your documents, let me know! We will then move to Phase 7: Evaluation System, where we start the "real" engineering work of measuring how accurate those answers actually are

.  
I have completed Phase 6, let's move to Phase 7.

How do I create the test dataset for Ragas evaluations?

What is the difference between faithfulness and answer relevance?

I have completed Phase 6, let's move to Phase 7.

Congratulations on successfully completing Phase 6! You now have a functional RAG system that generates answers based on uploaded documents

. Now, we are entering the most critical part of the project: Phase 7: Evaluation System

.

This is the phase that transforms your project from a "basic chatbot" into a production-grade AI application

.

## 1. What We Are Building

In this phase, we are building a "grading system" for your AI

. Instead of just reading the AI's answer and guessing if it's correct, we will use a framework called Ragas to automatically measure the quality of the system

. We will create a test dataset of question-answer pairs and run them through an evaluation engine to generate objective scores

.

## 2. Why It Matters

In the real world, AI teams don't just "hope" their AI works; they measure it

. This phase demonstrates AI Engineering and Performance Optimisation—skills that are highly sought after by recruiters in 2026

. By generating metrics, you can prove how reliable your system is and identify exactly where it might be failing (e.g., is it retrieving the wrong information or is the AI just hallucinating?)

.

## 3. Step-by-Step Implementation

Step 1: Install the Evaluation Framework We will use Ragas, which is the industry-recognised framework specifically built for RAG evaluation

.

```
pip install ragas
```

(Note: While our frontend is Next.js, Ragas is typically run as a Python-based service or via an API. For this project, you can create a simple Python script or a dedicated API route to handle these calculations.)

Step 2: Create a Test Dataset (Ground Truth) To grade the AI, you need a "gold standard" to compare it against

.

Select a document you have already uploaded.

Create 5–10 question-answer pairs based on that document.

For each pair, you need:

The Question.

The Ground Truth (the "perfect" answer you expect).

Step 3: Define Your Evaluation Metrics You will configure Ragas to measure four key metrics

:

Faithfulness: Does the answer stay true to the retrieved context (no hallucinations)?

Answer Relevance: Does the answer actually address the user's question?

Context Precision: Did the system find the most relevant chunks in the top results?

Context Recall: Did the system find all the information needed to answer the question?

Step 4: Run the Evaluation Pipeline Create a script that:

Takes a question from your test set.

Runs your Retrieval (Phase 5) and Generation (Phase 6) logic.

Passes the Question, the Retrieved Chunks, the AI Answer, and the Ground Truth into Ragas.

Calculates the scores for the four metrics above.

Step 5: Store the Scores in Supabase Create an evaluations table in your Supabase PostgreSQL database to store these results

.

```
create table evaluations (  
  id uuid primary key default uuid_generate_v4(),  
  question text,  
  ground_truth text,  
  generated_answer text,  
  faithfulness float,  
  relevance float,  
  precision float,  
  recall float,  
  created_at timestamp with time zone default now()  
);
```

#### 4. Code Logic Explanation

Ragas works by using a "Critic LLM" (usually GPT-4 or GPT-5) to act as a judge

.

To measure Faithfulness, Ragas breaks the AI's answer into individual claims and checks each one against the retrieved document chunks

.

To measure Context Precision, it checks if the "best" information was ranked at the very top of your search results

.

By using an LLM to "grade" another LLM, you get a scalable way to test thousands of documents without having to read them all yourself

.

## 5. Your Deliverable

A system that automatically generates scores (0.0 to 1.0) for every question in your test set and saves those scores into your Supabase database

.

Once you have successfully run an evaluation and can see the scores appearing in your evaluations table, let me know! We will then move to Phase 8: Experiment Engine, where we use these scores to find the "perfect" settings for your AI

.

How do I create the question-answer pairs for Ragas?

What is the Critic LLM in the evaluation process?

I have completed Phase 7, let's move to Phase 8.

How do I create the question-answer pairs for Ragas?

To create the question-answer pairs (often called a "test set" or "ground truth") for Ragas, you are essentially building a "gold standard" key that the system will use to grade the AI

.

In a production-grade project, there are two main ways to create these pairs: manually or synthetically.

### 1. The Manual Method (Best for Portfolios)

Since you are building this for your portfolio, creating 5–10 high-quality manual pairs is the best way to demonstrate you understand the data

.

Step A: Open one of the PDFs you uploaded in Phase 2

.

Step B: Find a specific fact in the text (e.g., "The warranty period is 24 months").

Step C: Write a Question based on that fact: "How long is the warranty period?"

Step D: Write the Ground Truth (the perfect answer): "The warranty period is 24 months."

Step E: Note which Context (the text chunk) contains the answer. Ragas needs this to measure Context Recall

.

## 2. The Synthetic Method (The "Pro" Way)

Real AI teams often have thousands of documents and cannot write questions by hand. Instead, they use an LLM (like GPT-5) to generate them

.

You can write a script that takes your document chunks

and sends them to OpenAI with a prompt like: "Given this text chunk, generate 3 professional questions and 3 accurate answers that can be found within it."

This is called Synthetic Data Generation. Including a script that does this automatically will highly impress recruiters as it shows Production AI Development skills

.

### What Each Pair Must Include

For Ragas to work correctly in your Evaluation Engine, every item in your test set should have these four parts

:

Question: What the user would ask.

Context: The actual text chunks from your database that contain the answer.

Answer: What your AI system generated (this is what you are grading).

Ground Truth: The "perfect" answer you (or the LLM) wrote in the steps above.

### Why This is Important for Phase 7

By creating these pairs, you allow the Evaluation System to calculate the metrics that recruiters care about

:

Faithfulness: Does the AI's answer match the Context?

Context Precision: Did the system find the Ground Truth in the top search results?

Next Step: I recommend manually creating 5 pairs for one of your documents first. Once you have those, you can insert them into your new evaluations table in Supabase to begin the grading process

.

How can I automate the synthetic generation of test pairs?

What are the specific schema requirements for the evaluations table?

I have the pairs ready, let's move to Phase 8.

What is the Critic LLM in the evaluation process?



In the context of the evaluation process for this platform, the Critic LLM is a high-reasoning language model (such as GPT-4 or GPT-5) that acts as an automated judge or "grader" for your AI system

,

.

While the term "Critic LLM" specifically refers to the role the model plays within the Ragas framework, its function in your project is to provide objective scores for the following areas:

**Automated Grading:** Instead of a human manually checking every response, the Critic LLM uses its reasoning capabilities to compare the generated answer against the retrieved context and the "ground truth"

.

**Metric Calculation:** It is responsible for calculating specific scores (from 0.0 to 1.0) for metrics such as Faithfulness (checking for hallucinations) and Answer Relevance

,

.

**Scalability:** By using an "LLM-as-a-judge" approach, the Critic LLM allows you to test thousands of documents and questions automatically, which is a standard practice in Production AI Development

,

.

In your current roadmap, this "Critic" role is what turns your simple answering system into a measurable engineering project by populating the Metrics Dashboard with data-driven results

,

.

How do I automate Phase 7 with a Python script?

Show me the table structure for storing Ragas scores.

I have completed Phase 7, let's move to Phase 8.

I have completed Phase 7, let's move to Phase 8.

With Phase 7 complete, you now have the ability to objectively "grade" your AI. Now we move to Phase 8: Experiment Engine, which is where you use those grades to find the absolute best version of your platform

.

## 1. What We Are Building

In this phase, we are building a framework to run "science experiments" on your RAG pipeline

. Instead of having just one fixed way of processing documents, you will test different settings—such as different chunk sizes (200 vs. 500 vs. 1000 characters) or different prompt variations—and compare their scores

.

## 2. Why It Matters

In professional AI development, you never know which settings will work best for a specific type of document until you test them

. By building an experiment engine, you are demonstrating Experiment Design and Performance Optimisation

.

Recruiters want to see that you didn't just pick random settings; they want to see a Leaderboard that proves you chose your final configuration because it had the highest Faithfulness and Relevance scores

.

## 3. Step-by-Step Implementation

Step 1: Create the Experiments Table You need a place to store the "average scores" for each configuration you test. Run this in your Supabase SQL Editor

:

```
create table experiments (  
  id uuid primary key default uuid_generate_v4(),  
  name text, -- e.g., "Chunk Size 200"  
  chunk_size int,  
  embedding_model text,  
  retrieval_method text,  
  avg_faithfulness float,  
  avg_relevance float,  
  avg_precision float,  
  avg_recall float,  
  created_at timestamp with time zone default now()  
);
```

Step 2: Set Up the "Loop" Script Create a script (or a background job) that automates the following for each experiment (e.g., for a 200-character chunk size, then a 500-character chunk size)

:

Reprocess: Re-run Phase 3 (Chunking) and Phase 4 (Embeddings) using the new settings.

Evaluate: Re-run your Phase 7 (Evaluation) test set against this new data.

Aggregate: Calculate the average score for all questions in that test set.

Save: Record the results in your new experiments table.

Step 3: Build the Experiment Leaderboard In your Next.js frontend, create a new page (/dashboard/experiments) that fetches data from the experiments table

.

Use Shadcn UI tables to display the configurations side-by-side

.

Highlight the "Winner": Visually mark the configuration that achieved the highest overall score.

#### 4. Logic Explanation

Before you start coding, understand the "Control Variable" logic:

The Variable: In your first experiment, the only thing you should change is the Chunk Size (e.g., 200 vs 500 vs 1000)

. Keep the AI model and the prompt the same.

The Comparison: By comparing the Ragas scores for these three sizes, you might discover that 500-character chunks are perfect, while 1000-character chunks make the AI lose focus, resulting in lower Context Precision

.

The Data Loop: This is a "closed-loop" system. The scores from Phase 7 tell the Experiment Engine in Phase 8 which version is better, which then informs how you should set up your final production system

.

#### 5. Your Deliverable

A Leaderboard in your dashboard that lists different experiment runs and clearly shows which RAG configuration performed the best based on your Ragas metrics.

.

Once you have run at least two different experiments (e.g., comparing two different chunk sizes) and can see them on your leaderboard, let me know, and we will move to Phase 9: Observability!

I've completed Phase 8, let's move to Phase 9.

How do I compare chunk sizes in the leaderboard?

What does Langfuse do for observability in Phase 9?

How do I compare chunk sizes in the leaderboard?

Comparing chunk sizes in the leaderboard is the core objective of Phase 8 (Experiment Engine). It allows you to move beyond guessing and use data to determine which settings make your AI the most accurate

.

To compare them effectively, you should follow these steps:

### 1. Run Parallel Experiments

You will process the same document multiple times using different chunk sizes, such as 200, 500, and 1000 characters

. For each size, you must:

Re-chunk the text (Phase 3)

.

Generate new embeddings for those specific chunks (Phase 4)

.

Store them separately in your database so you can query them independently

.

### 2. Use a Consistent Test Dataset

To make the comparison "fair," you must run the exact same set of question-answer pairs (from Phase 7) against each different chunk size

. This ensures that any change in the final score is caused by the chunk size, not by the difficulty of the questions.

### 3. Aggregate Ragas Metrics

For every chunk size experiment, your Evaluation Engine (Ragas) will generate scores for four key metrics

:

Faithfulness: Does the larger chunk size help the AI stay more truthful?

Answer Relevance: Does a smaller chunk make the answer too brief or missing detail?

Context Precision: Does the system find the right information more easily with specific chunk sizes?

Context Recall: Does a larger chunk provide more of the "gold standard" answer?

### 4. Display the Leaderboard

In your Next.js dashboard, you will create a professional table or chart using Shadcn UI to display these results side-by-side

.

A typical leaderboard row for an experiment might look like this:

Experiment Name

Chunk Size

Avg. Faithfulness

Avg. Relevance

Avg. Precision

Small Chunks

200

0.82

0.75

0.91

Medium Chunks

500

0.89

0.88

0.85

Large Chunks

1000

0.85

0.82

0.70

Why This Impresses Recruiters

By showing this comparison, you demonstrate Performance Optimisation and Experiment Design

. It proves to a recruiter that you understand that RAG systems are not "set and forget"; they require constant testing and data-driven adjustments to reach production quality

.

What is Phase 9: Observability?

How do I integrate Langfuse into the pipeline?

What should I record in the tracing dashboard?

I've completed Phase 8, let's move to Phase 9.

With Phase 8 complete, you've used data to find the best settings for your AI. Now we move to Phase 9: Observability, which is the final technical layer of your production architecture.

## 1. What We Are Building

In this phase, you are installing a "black box flight recorder" for your AI called Langfuse.

You will build a system that records every single detail of an AI interaction, including:

The original Question.

The exact Retrieved Chunks and their Similarity Scores.

The final Prompt sent to the LLM.

The AI's Answer.

The Evaluation Score (Faithfulness, Relevance, etc.) generated in Phase 7.

## 2. Why It Matters

In production, you can't just "hope" the AI is doing the right thing. Recruiters love seeing observability because it shows you know how to trace, debug, and monitor a live system.

If a user gets a bad answer, a tracing dashboard allows you to look back and see exactly where the failure happened—was the retrieval bad, or did the LLM simply ignore the instructions?

## 3. Step-by-Step Implementation

**Step 1: Set Up Langfuse** Langfuse is an open-source observability platform.

Create a free account at [Langfuse.com](https://langfuse.com).

Create a new project and generate your Public Key, Secret Key, and Host URL.

Add these to your `.env.local` file.

**Step 2: Install the Langfuse SDK** Run this in your terminal to allow your Next.js app to talk to Langfuse:

```
npm install langfuse
```

**Step 3: "Wrap" Your RAG Pipeline** You will update your search and answer logic (from Phases 5 and 6) to send data to Langfuse. Instead of just running the functions, you will create a "Trace" for each user question.

**Start a Trace:** When the user clicks "Search," create a new trace object.

**Log Retrieval:** Create a "span" (a sub-task) for the retrieval step, logging the chunks you found in Supabase.

**Log Generation:** Create another "span" for the OpenAI call, logging the exact prompt and the resulting answer.

**Step 4: Connect Evaluation Scores** This is the most advanced part: Take the Ragas scores from Phase 7 and send them to Langfuse as "scores" for that specific trace.

This allows you to see the accuracy of every single interaction directly in your observability dashboard.

## 4. Code Logic Explanation

Before you implement, understand how Tracing works:

The Trace: This is the "parent" container for one full interaction (Question → Answer).

Spans: These are the "steps" inside. One span for "Retrieval" and one span for "Generation."

Metadata: You can attach the user's ID or the experiment ID (from Phase 8) to the trace. This lets you filter your logs to see, for example, "Only show me logs where we used 200-character chunks."

## 5. Your Deliverable

A tracing dashboard in Langfuse where you can click on any previous question and see the full step-by-step "thought process" of your AI, including its accuracy scores.

Once you have successfully logged an interaction and can see the trace in your Langfuse dashboard, let me know! We will then move to the final Phase 10: Portfolio Polish, where we turn all this hard work into a recruiter-ready showcase.

How do I create the professional charts and metrics dashboard?

What should I include in the architecture diagram for recruiters?

How do I write a compelling case study for this project?

How do I create the professional charts and metrics dashboard?

To create a professional metrics dashboard for your portfolio, you are moving into Phase 10: Portfolio Polish

. This is where you transform raw data from your database into a visual story that proves your system's performance to recruiters

.

Here are the steps to build your "Senior AI Engineer" dashboard:

### 1. What We Are Building

You are creating a central hub in your Next.js app that visualizes the "health" and "intelligence" of your RAG system

. Instead of showing a simple chat history, you will display:

Performance Trends: How accuracy changes over time.

Evaluation Radar: A specialized chart showing the balance between your four Ragas metrics (Faithfulness, Relevance, Precision, and Recall)

.

The Experiment Leaderboard: A direct comparison of different configurations, such as chunk sizes

.

### 2. Why It Matters

Recruiters in 2026 are looking for AI Engineering and Performance Optimisation skills

. A dashboard with professional charts demonstrates that:

You don't just "hope" the AI works; you measure it

.

You understand the trade-offs in RAG systems (e.g., how larger chunks might increase recall but decrease precision)

.

You can communicate complex technical data to stakeholders visually

.

### 3. Implementation Instructions

Step 1: Install a Charting Library While Shadcn UI provides the "look and feel" (containers, cards, and tables), you need a library to draw the actual lines and bars

. Recharts is the industry standard for Next.js:

```
npm install recharts
```

Step 2: Fetch Data from Supabase You need to pull the scores you saved in Phase 7 and Phase 8

. In your Next.js page.tsx, create a function to fetch:

Averages: The overall average of your Ragas metrics.

History: The last 20 evaluation runs to show progress.

Experiments: The data from your experiments table to populate the leaderboard

.

Step 3: Create the "RAG Health" Radar Chart A Radar Chart (or Spider Chart) is perfect for RAG evaluation

. It has four points: Faithfulness, Relevance, Context Precision, and Context Recall

.

If the shape is large and balanced, your system is "production-ready."

If the "Precision" point is low, it tells a recruiter you need to work on your retrieval settings

.

Step 4: Build the Experiment Leaderboard Use a Shadcn UI Table to list your experiment runs

.

Columns: Configuration (e.g., "Chunk Size: 500"), Embedding Model, and the four Ragas scores

.

Visual Cue: Use a green badge for the highest-scoring row so it's immediately obvious which configuration "won" the experiment



#### 4. Logic and Code Explanation

Data Aggregation: Before sending data to the charts, you often need to calculate averages in your code. For example, your dashboard shouldn't show every single question; it should show the Average Faithfulness across the entire document

Client-Side Rendering: Because charts are interactive (hovering to see values), you will mark your dashboard components with 'use client' in Next.js

Visual Hierarchy: Use Shadcn Cards to highlight "Key Performance Indicators" (KPIs) at the top of the page—big numbers showing your overall accuracy percentage—before showing the detailed charts below

#### 5. Your Deliverable

A "Project Showcase" dashboard that features at least one Radar Chart (for Ragas metrics), one Line Chart (for performance over time), and the Experiment Leaderboard table

This dashboard is the "crown jewel" of your portfolio. Once you have these visuals live, your project is officially Portfolio-Ready

. Would you like to discuss how to write the final "Case Study" to accompany these charts?

How should I write the final case study?

How do I show the architecture diagram?

What screenshots are most important for recruiters?

What should I include in the architecture diagram for recruiters?

To impress recruiters in 2026, your architecture diagram must move beyond a simple "Chatbot" flow and instead visualize a closed-loop production system

. Recruiters are looking for evidence of AI Engineering, Observability, and Performance Optimisation

Here is exactly what you should include in your diagram to demonstrate a professional, production-ready architecture:

##### 1. The Multi-Stage Data Pipeline

Show how a raw file becomes searchable data. Instead of one block for "Upload," break it down to show you understand the engineering steps

:

Document Parser & Chunker: Use LlamaIndex to show how you handle PDF extraction and "Sentence Splitting"

.

Embedding Engine: Explicitly label OpenAI (text-embedding-3-small) as the model used to create vectors

.

Vector Database: Represent Supabase (pgvector) as the storage for both metadata and embeddings

.

## 2. The Retrieval & Generation Flow

This is the "active" part of your system where the user interacts

:

Retriever: Show the query flow from the user into the Supabase Vector Database

.

Prompt Template Engine: Include a block for the "Grounded Prompt" where retrieved chunks are injected

.

LLM (Generator): Label this as OpenAI (GPT-4 or GPT-5) to show you are using high-reasoning models for final output

.

## 3. The "AI Engineering" Layer (The Differentiator)

This is what sets your project apart from "tutorial" apps. You must include these three components to prove it is a production system

:

Evaluation Engine (Ragas): Show a branch where the AI Answer, the Retrieved Context, and a Ground Truth are sent to Ragas for scoring

.

Observability & Tracing (Langfuse): Draw a layer that wraps around the entire process (Retrieval + Generation) to show every interaction is being recorded

.

The Experiment Engine: Show a feedback loop where different settings (like Chunk Sizes) are tested, scored, and then displayed on a Leaderboard

.

## 4. Professional Tech Stack Branding

Recruiters recognize industry-standard tools. Include the logos or names of the following in their respective sections

:

Frontend: Next.js & Shadcn/UI (for the professional "enterprise" look)

.

Backend/Auth: Supabase

.

Orchestration: LlamaIndex

.

Evaluation: Ragas

.

Monitoring: Langfuse

.

Summary of the Flow for Your Diagram:

Ingestion: User → Next.js → Supabase Storage → LlamaIndex Parser → OpenAI Embeddings → Supabase Vector DB

.

Inference: User Question → Vector Search → Context Retrieval → GPT-4/5 Generation → Answer

.

Validation: Answer + Context → Ragas Evaluation → Supabase Metrics Table

.

Monitoring: All steps → Langfuse Tracing

.

Including these elements demonstrates that you aren't just building a "toy," but a system designed for reliability, measurement, and continuous improvement

.

Shall we discuss the case study write-up?

How can I explain these metrics during an interview?

Which charts are best for showing RAG performance?

Which charts are best for showing RAG performance?

To effectively demonstrate RAG performance to recruiters, you should focus on visualisations that highlight accuracy, reliability, and data-driven optimisation

,

. Based on the project's architecture and the Ragas framework, here are the best charts to include in your Metrics Dashboard

,

:

### 1. Radar Chart (Spider Chart)

This is arguably the most important chart for a RAG project. It is used to show the balance between the four core Ragas metrics

,

:

Faithfulness: Measures if the answer is grounded in the retrieved context (no hallucinations)

,

.

Answer Relevance: Measures how well the answer addresses the user's question

,

.

Context Precision: Measures if the most relevant chunks are ranked highly in retrieval

,

.

Context Recall: Measures if all necessary information was found to answer the question

,

.

Why it works: A balanced, large shape on the radar chart immediately signals a "production-ready" system to a recruiter, while an unbalanced shape identifies exactly where the system needs tuning.

### 2. Line Chart (Performance Trends)

Use a line chart to track your evaluation scores over time

.

What to show: Plot the average Ragas scores across different versions or updates of your application.

Why it works: It demonstrates Performance Optimisation and shows that you are actively monitoring the system to ensure accuracy doesn't degrade as you add more documents

,

.

### 3. Experiment Leaderboard (Comparison Table)

While technically a table, a leaderboard is a vital visual component for showing Experiment Design

,

.

What to show: Compare different Chunk Sizes (200 vs. 500 vs. 1000), different Embedding Models, or various Prompt Variations

.

Why it works: It proves you didn't just pick settings at random. By highlighting the "winning" configuration with a high average score, you show you used a scientific approach to build the best possible version of the platform

,

.

### 4. Bar Charts (Direct Comparisons)

Bar charts are excellent for side-by-side comparisons of specific experiments

.

What to show: You might show a bar chart comparing the "Total Hallucination Rate" (inverse of faithfulness) between two different LLMs, such as GPT-4 vs. GPT-5

,

.

Why it works: It provides a clear, high-impact visual of how specific technical choices (like switching models) directly affect the quality of the AI's output.

By using Next.js with Shadcn UI and a library like Recharts, you can build these professional-grade visuals to turn your project into a high-impact portfolio piece

,

.

How should I write the final case study?

Which tools help create the architecture diagram?

Can I add a human-in-the-loop feedback feature?

so each useraccount will need dashboard to show results , and able to unupload docs... is this right or am i wrong we arent building this we are just building a app that uploads docs and answers questions basically im confused

You are partially right, but there is a major difference that makes this project special.

If you just build an app that uploads docs and answers questions, you have built a standard "chatbot" that thousands of other developers have already made

. To impress recruiters in 2026, you are building a Production-Grade Evaluation Platform

.

Here is the breakdown to clear up the confusion:

1. Yes, it uploads docs and answers questions

That is the "engine" of the app. You need this part so there is something to actually measure

. This is covered in:

Phase 2: Uploading the PDFs

.

Phase 6: Getting the AI to answer based on those PDFs

.

2. BUT, the real goal is the "Scoreboard" (Metrics & Evaluation)

The reason you have a dashboard and user accounts isn't just to chat; it's to prove the AI is working correctly

. In a real company, an engineer doesn't just say "the AI seems smart." They say "our AI has a 92% Faithfulness score"

.

Your dashboard will show:

Evaluation Scores: Automatic grades (from Ragas) that tell you if the AI is hallucinating or telling the truth

.

Experiments: A way to test if the AI gets smarter if you change the "chunk size" of the documents

.

Observability: A "flight recorder" (via Langfuse) that tracks every single step the AI took to get to an answer

.

3. Why the "Dashboard" is the most important part

The source material explicitly states: "This is NOT a chatbot project"

. Most portfolio projects stop at the answer. Yours keeps going:

Standard Project: Upload → Chat → Answer

. Your Project: Upload → Answer → Evaluation → Metrics → Experimentation

.

## Summary

You aren't just building a tool to read PDFs; you are building a scientific laboratory for AI

. The user account and dashboard are there so an engineer (the user) can upload documents, run tests, and see a professional chart proving which settings make the AI the most accurate

.

That is what demonstrates AI Engineering and Performance Optimisation to a recruiter

.

How do I write the final case study?

What screenshots are most important for my portfolio?

Show me how to add the final performance comparisons.

I see so the only user is me the dev, and the output is from the pdf docs i upload, whatever subject? its more about showing it can be built, how & depth or quality of results

Exactly. You have hit the nail on the head. While a regular user could use this to chat with a PDF, the true "user" of this platform is a Senior AI Engineer (you) or a Technical Recruiter looking at your work

.

Here is the breakdown of why the depth and quality of the engineering matter more than the subject of the PDFs:

### 1. The Subject is Irrelevant, the "Grounding" is Everything

You can upload a PDF about medieval history, medical insurance, or Pokémon. The AI's job is to answer questions only using that specific document

.

The Goal: To prove that your RAG (Retrieval-Augmented Generation) pipeline is so well-built that it can handle any subject with high accuracy and zero hallucinations

.

### 2. Showing How It Was Built

Recruiters in 2026 aren't just looking for "someone who can use an API." They want someone who understands the Production AI Pipeline

. By following the roadmap, you are showing them:

Architectural Depth: You didn't just build a simple flow; you built a system with Document Parsers, Chunkers, and Vector Databases

.

Observability: You included Langfuse because you know that real-world AI systems need to be traced and monitored

Professional Stack: Using Next.js, Supabase, and LlamaIndex shows you use the industry-standard tools that companies actually use

### 3. Depth and Quality of Results (The "Science")

This is the "NOT a chatbot" part

The most important results in this project aren't the AI's answers—they are the Evaluation Metrics and Experiment Results

:

Evaluation: You are proving the quality of the results by using Ragas to generate objective scores for things like "Faithfulness" and "Context Precision"

Experimentation: You are showing that you tested different settings (like chunk sizes) to find the "perfect" configuration

The Dashboard: Your final deliverable is a Leaderboard and Charts that show a recruiter: "I tested 3 different ways to build this, and here is the data that proves this one is the best"

In short: The PDF content is just the "test data." The real "product" you are showing off is your ability to design, measure, and optimize a complex AI system

What kind of PDFs work best for the test dataset?

How should I explain the Ragas scores to recruiters?

Which experiment should I try first after chunk sizes?

Which experiment should I try first after chunk sizes?

Based on the development roadmap for Phase 8, once you have compared chunk sizes, you should experiment with Embedding Models, Retrieval Methods, or Prompt Variations

To demonstrate the most growth in your "AI Engineering" skills, here are the three experiments you should consider next:

#### 1. Prompt Variations (Highly Recommended)

This experiment focuses on the "Generation" part of your RAG pipeline.

What to test: Create different versions of your Prompt Template



. For example, compare a simple prompt against one that includes "Chain of Thought" instructions (e.g., "Think step-by-step before answering").

The Goal: See which prompt variation results in the highest Faithfulness and Answer Relevance scores in Ragas

. This proves you can "engineer" the AI's behavior through better instructions.

## 2. Retrieval Methods

This experiment looks at how you find information in the Supabase Vector Database

.

What to test: Experiment with different "Top-K" values. For instance, compare retrieving the top 3 chunks versus the top 7 chunks

.

The Goal: Determine the "sweet spot" for context. Too little context might lower your Context Recall, while too much might "confuse" the AI and lower Context Precision

.

## 3. Embedding Models

This is a deeper architectural experiment involving the mathematical foundation of your search

.

What to test: Compare the current text-embedding-3-small model against a larger or different model

.

The Goal: Measure if a different model improves the similarity scores and overall Context Precision, showing you can optimize the very core of the retrieval process

.

Which one would you like to implement first? I recommend Prompt Variations as it often yields the most immediate and visible improvements to your Evaluation Engine results

.

Let's go with Prompt Variations

How do I test different Retrieval Methods?

Show me how to compare Embedding Models

Let's go with Prompt Variations

Choosing to experiment with Prompt Variations is a sophisticated next step. In the world of AI Engineering, this is often where the biggest gains in accuracy are found without changing a single line of database code

.

## 1. What We Are Building

In this part of the Experiment Engine, we are creating a "A/B Test" for your AI's instructions. You will define two or more different Prompt Templates—the sets of instructions given to the LLM (GPT-4 or GPT-5)—and run your evaluation set against each one to see which version produces the highest scores

.

## 2. Why It Matters

This experiment demonstrates Performance Optimisation and Experiment Design to recruiters

. It proves that you understand that an LLM's performance is highly dependent on how instructions are framed. By showing a leaderboard where "Prompt B" outperformed "Prompt A," you demonstrate that you can scientifically "tune" an AI's behavior to be more faithful and relevant

.

## 3. Step-by-Step Implementation

Step 1: Define Your Variations Create at least two distinct templates in your code. For example:

Template A (Simple): "Answer the question using the context provided."

Template B (Chain-of-Thought): "You are an expert analyst. First, extract the relevant facts from the context. Then, step-by-step, use those facts to answer the user's question. If the context does not contain the answer, explicitly state that you cannot find the information."

Step 2: Update the Experiment Logic In your experiment script (from Phase 8), add a new loop that:

Iterates through each prompt template.

Uses the same retrieved context from a fixed chunk size (e.g., the 500-character "winner" from your last experiment) to keep the test fair.

Sends the question and context to the LLM using the current template.

Step 3: Evaluate with Ragas For each template run, pass the generated answers to the Evaluation Engine (Ragas)

. This will generate scores for:

Faithfulness (Did the "Chain-of-Thought" prompt reduce hallucinations?)

.

Answer Relevance (Did the "Expert Analyst" persona provide more helpful answers?)

.

Step 4: Record Results in Supabase Insert a new row into your experiments table for each prompt version.

Name: "Prompt: Simple" vs. "Prompt: Chain-of-Thought".

Average Scores: Store the resulting Ragas metrics

.

#### 4. Code Logic Explanation

Think of the Prompt Template as the "envelope" that carries your data to the LLM

.

The Placeholder: Your code will look for markers like {context} and {question} and swap them out for the real data retrieved from Supabase

.

The Instruction: By adding "Chain-of-Thought" instructions (asking the AI to think step-by-step), you are essentially giving the LLM more "computational time" to process the context before it starts writing the final answer.

The Control: By keeping the retrieval method and chunk size exactly the same, you ensure that any change in the Ragas scores is purely a result of the different instructions

.

#### 5. Your Deliverable

An updated Experiments Leaderboard in your dashboard that compares your prompt variations and highlights the "Winning Template" based on its superior evaluation scores

.

Once you have run these evaluations and can see which prompt style makes your AI more accurate, let me know! We will then be ready for the final technical hurdle, Phase 9: Observability, to start "tracing" these interactions in real-time

.

How do I code the prompt experiment loop?

What is the benefit of Chain-of-Thought prompting?

Can we use a Critic LLM to score these?

What is the benefit of Chain-of-Thought prompting?

Based on our conversation history and the project roadmap, Chain-of-Thought (CoT) prompting is a technique where the AI is instructed to "think step-by-step" before arriving at a final answer.

The specific benefits of using this method within your AI Knowledge Base Evaluation Platform include:

Increased "Computational Time" for the LLM: By requiring the model to extract facts and reason through them before writing the final response, you are effectively giving the AI more space to process the retrieved context from your documents.

Improved Accuracy Metrics: In a RAG system, CoT can lead to significantly higher scores in your Evaluation Engine

. Specifically, it helps improve:

Faithfulness: It reduces the likelihood of hallucinations by forcing the AI to ground its reasoning in the specific chunks retrieved from your Supabase Vector Database

.

Answer Relevance: It often results in more comprehensive and helpful answers because the AI has systematically addressed the components of the user's question.

Proof of Performance Optimisation: For your portfolio, implementing a CoT experiment demonstrates to recruiters that you can scientifically "tune" AI behaviour

. It allows you to show a data-driven comparison on your Leaderboard

, proving that more sophisticated instructions can lead to better production outcomes than simple prompts.

In your current Experiment Engine phase

, the benefit of CoT is that it provides a clear, measurable variable to test against a standard prompt, allowing you to visually demonstrate the quality and depth of your AI engineering to a recruiter

.

How do I set up the Chain-of-Thought experiment?

How should I record these prompt scores in Supabase?

Can we move to Phase 9: Observability now?

Can we move to Phase 9: Observability now?

Yes, we can now move to Phase 9: Observability.

As your Senior AI Engineer, I'll guide you through setting up a professional "black box flight recorder" for your AI system. This is the final technical layer that proves your application is ready for real-world usage.

## 1. What We Are Building

In this phase, we are integrating Langfuse, an open-source observability platform designed specifically for AI

. You will build a system that records the "trace" of every single interaction, capturing:

The original Question.

The exact Retrieved Chunks and their Similarity Scores.

The full Prompt sent to the LLM (including your new Chain-of-Thought instructions).

The AI's Answer.

The Evaluation Scores (Faithfulness, Relevance, etc.) that you generated in Phase 7

.

## 2. Why It Matters

In a production environment, you cannot simply guess why an AI gave a poor answer. Recruiters love seeing observability because it demonstrates that you know how to trace, debug, and monitor a live system

.

By having a tracing dashboard, you can "look under the hood" of any past interaction. If a user receives a low-quality answer, you can pinpoint exactly where the failure occurred: was it poor retrieval from the database, or did the LLM fail to follow your prompt instructions

?

## 3. Step-by-Step Implementation

### Step 1: Set Up Langfuse

Create a free account at [Langfuse.com](https://langfuse.com).

Create a new project and generate your Public Key, Secret Key, and Host URL.

Add these keys to your `.env.local` file so your Next.js app can securely communicate with them

.

Step 2: Install the SDK Install the Langfuse library to start sending data from your application:

```
npm install langfuse
```

Step 3: "Wrap" Your RAG Pipeline Update your search and answer logic (from Phases 5 and 6) to send data to Langfuse. Instead of just running functions, you will create a "Trace" for each question:

Start a Trace: When a user submits a question, initialize a new trace object.

Log Retrieval: Create a "span" (a sub-step) for the retrieval process, logging the chunks found in your Supabase Vector Database

.

Log Generation: Create a second "span" for the OpenAI call, logging the exact prompt and the resulting answer

.

Step 4: Attach Evaluation Scores Link the Ragas scores from Phase 7 to the specific trace in Langfuse. This allows you to see the accuracy of every interaction directly within your observability dashboard

.

## 4. Code Logic Explanation

Before you implement the code, it's important to understand the hierarchy of Tracing:

The Trace: This is the "parent" record for one full interaction (from Question to final Answer)

.

Spans: These are the individual "steps" or "chapters" inside a trace. You will have one span for Retrieval (fetching data from Supabase) and one for Generation (calling OpenAI)

.

Metadata: You can attach extra info, such as the `experiment_id` from Phase 8. This allows you to filter your logs in Langfuse to see, for example, "only show me traces where we used the 200-character chunk size"

.

## 5. Your Deliverable

A live tracing dashboard in Langfuse where you can click on any previous user question and see the full, step-by-step reasoning process of your AI, complete with its accuracy scores

.

Once you have successfully logged an interaction and can see the trace in your Langfuse dashboard, let me know, and we will move to the final Phase 10: Portfolio Polish!

How do I connect Langfuse to my Next.js app?

What is a 'span' in an AI trace?

I have completed Phase 9, let's move to Phase 10.

How do I connect Langfuse to my Next.js app?

Connecting Langfuse to your Next.js app is the core task of Phase 9: Observability. This step ensures every part of your AI's "thought process"—from the initial question to the final evaluation score—is recorded for debugging and performance monitoring

.

Following the production-grade architecture of this project, here is how you connect it

:

### 1. Setup Environment Variables

First, you must securely store your credentials. Create a free account on the Langfuse website and generate your API keys. Add these to your `.env.local` file:

```
LANGFUSE_PUBLIC_KEY=pk-lf-...
```

```
LANGFUSE_SECRET_KEY=sk-lf-...
```

```
LANGFUSE_HOST=https://cloud.langfuse.com # or your specific region
```

### 2. Install the Langfuse SDK

In your terminal, install the official web SDK to allow your Next.js backend to communicate with the observability platform

:

npm install langfuse

### 3. Initialize the Langfuse Client

Create a utility file (e.g., `src/lib/langfuse.ts`) to initialize the client. This ensures you aren't creating a new connection every time a user asks a question:

```
import { Langfuse } from "langfuse";

export const langfuse = new Langfuse({
  publicKey: process.env.LANGFUSE_PUBLIC_KEY,
  secretKey: process.env.LANGFUSE_SECRET_KEY,
  baseUrl: process.env.LANGFUSE_HOST,
});
```

### 4. Implementation: "Wrapping" the Pipeline

To record an interaction, you must wrap your existing RAG logic (from Phases 5 and 6) in a Trace. Here is the logical flow you will implement in your API route

:

**Start a Trace:** Initialize a trace at the beginning of the user's request. Give it a name like "Document QA".

**Create a Retrieval Span:** Before you query Supabase, start a "span" (a sub-step). Log the retrieved chunks and their similarity scores here

.

**Create a Generation Span:** Start a second span for the OpenAI call. Log the final prompt (including your Chain-of-Thought instructions) and the generated answer

.

**Log the Evaluation:** Finally, attach the scores from your Ragas evaluation (Faithfulness, Relevance, etc.) to the trace so you can see the quality of that specific answer in the dashboard

.

### 5. Why This Implementation Matters

By connecting Langfuse this way, you move beyond a simple "input-output" app. Recruiters love seeing this because it demonstrates:

**Production Readiness:** You can debug exactly why an AI gave a "bad" answer by looking at the specific chunks retrieved

.

**Data-Driven Engineering:** You are linking Evaluation Scores directly to live traces, allowing you to see if your "Chain-of-Thought" prompt variation actually improved accuracy in real-world scenarios

.

Once you have successfully logged your first trace and can see the "spans" for Retrieval and Generation in your Langfuse dashboard, you will have completed the technical requirements for Phase 9

.

How do I start a trace in my API route?

Can we log the Ragas scores directly to Langfuse?

What is the difference between a Trace and a Span?

I have completed Phase 9, let's move to Phase 10.

Congratulations on completing Phase 9! You have successfully built a sophisticated AI system with full observability and tracing

. Now, we move to the final and most important step for your career: Phase 10: Portfolio Polish

.

## 1. What We Are Building

In this final phase, you are transforming your technical backend into a visual narrative for recruiters

. You will build:

Professional Dashboard: A centralized "Mission Control" using Shadcn UI

.

Performance Charts: Visual representations of your Ragas metrics (Faithfulness, Relevance, etc.)

.

Experiment Leaderboard: A direct comparison of different configurations (like chunk sizes or prompt variations)

.

Architecture Diagram: A clear map of your production-grade system

.

Case Study Write-up: A professional explanation of the problem you solved and the technical decisions you made

.

## 2. Why It Matters

This is what differentiates a "tutorial project" from a "production-ready application"

. Most developers show a simple chatbot; you are showing an Evaluation Platform



. This phase demonstrates the high-level skills recruiters are hiring for in 2026: AI Engineering, Performance Optimisation, and Experiment Design

.

### 3. Step-by-Step Implementation

Step 1: Build the Main Metrics Dashboard Using Next.js and Shadcn UI, create a dashboard that fetches your evaluation scores from Supabase

.

Key Metrics: Display big "KPI cards" at the top showing your average Faithfulness and Answer Relevance scores

.

Visual Charts: Use a library like Recharts to build:

Radar Chart: To show the balance between your four Ragas metrics

.

Line Chart: To show how your system's performance improved as you ran different experiments

.

Step 2: Create the Experiment Leaderboard Create a professional table that lists all the experiments you ran in Phase 8

.

Columns: Configuration Name (e.g., "CoT Prompt + 500 Chunk Size"), Embedding Model, and the resulting scores

.

The "Winner": Visually highlight the top-performing configuration to show you can identify the best approach based on data

.

Step 3: Design the Architecture Diagram Create a high-quality visual showing the data flow from User Upload through LlamaIndex processing to Supabase Vector Database and finally to the Evaluation Engine

.

Pro Tip: Include the Langfuse logo to show the observability layer wrapping the whole system

.

Step 4: Write the Case Study Don't just share a link to your code. Write a "README" or a blog post that explains:

The Challenge: Why standard RAG systems hallucinate.

The Solution: How you used Ragas and Experimentation to build a more reliable system

.

The Result: "I improved Faithfulness scores from 0.72 to 0.94 by optimizing chunk sizes and prompts."

#### 4. Concept Explanation

Data Visualization: This isn't just about "pretty pictures." It's about Data-Driven Engineering. By turning raw numbers in Supabase into charts, you make it easy for a non-technical recruiter or a busy hiring manager to see your value instantly

.

The "Grounded" Narrative: Your case study should focus on how you solved the hallucination problem

. This is the biggest pain point for companies using AI, and showing you can measure and solve it makes you highly employable

.

#### 5. Your Deliverable

A portfolio-ready project featuring a professional dashboard with live charts, a clear architecture diagram, and a case study that proves your AI system is accurate and reliable

.

Once your dashboard is live and your charts are showing your hard-earned metrics, you have officially finished the Production-Grade AI Knowledge Base Evaluation Platform! Please let me know if you would like me to create an artifact to help summarize your final results or if you would like to discuss how to present this on your CV.

How should I structure the final case study write-up?

Which charts should I use to show performance gains?

Can you help me summarize my final results for CVs?

Which charts should I use to show performance gains?

To show performance gains in your project, you should focus on visualisations that highlight the accuracy and reliability improvements achieved through your experiments with chunk sizes, prompts, and retrieval methods

.

Based on the sources and our previous discussion, here are the most effective charts to include in your Phase 10 Portfolio Polish dashboard:

##### 1. Radar Chart (Spider Chart) for Ragas Metrics

This is the most critical chart for showing the overall "health" of your RAG system

.

What it shows: It plots the four core Ragas metrics: Faithfulness, Answer Relevance, Context Precision, and Context Recall

.

Performance Gain: You can overlay two shapes—one for your "Baseline" (e.g., standard prompt) and one for your "Optimised" version (e.g., Chain-of-Thought prompt). The growth in the area of the shape visually represents the performance gain

.

## 2. Bar Charts for Experiment Comparisons

Bar charts are ideal for showing direct "head-to-head" improvements from your Experiment Engine

.

What it shows: The average scores for different configurations, such as Chunk Size 200 vs. 500 vs. 1000

.

Performance Gain: By placing these bars side-by-side, you clearly demonstrate which technical choice led to a higher score in Context Precision or Recall

.

## 3. Line Chart for Performance Trends

A line chart is perfect for showing how your system improved as you iterated through the project phases

.

What it shows: The y-axis represents the evaluation score (0.0 to 1.0), and the x-axis represents the "Run" or "Version"

.

Performance Gain: This chart tells the story of your development journey, showing how scores climbed as you moved from a basic RAG setup to an optimised, grounded system

.

## 4. The Experiment Leaderboard

While technically a table, this is a vital visual for recruiters to see Performance Optimisation in action

.

What it shows: A ranked list of your experiment runs (e.g., different embedding models or retrieval methods)

.

Performance Gain: It highlights the "Winner" of your experiments, proving you used a scientific approach to select the best-performing architecture for your production system

.

### Implementation Tip

You should build these using Shadcn UI components and a library like Recharts within your Next.js frontend to maintain a modern, enterprise-ready appearance that recruiters will recognise

. By showing these charts, you move beyond "chatting with PDFs" and demonstrate deep expertise in AI Engineering and Experiment Design

.

How should I write the final case study write-up?

Can you help me summarise my final project results?

What are the best ways to present this on my CV?

so basically the frontend needs homepage, login/signup? and dashboard with ability to show what graphs and results?

Yes, your understanding of the frontend structure is correct. To create a professional AI Knowledge Base Evaluation Platform that impresses recruiters, you will need a cohesive user journey from landing to deep technical analysis

.

Here is how the frontend should be structured and exactly what the dashboard should display:

#### 1. Essential Frontend Pages

Homepage: This serves as the "pitch" for your project, explaining that it is a production-grade evaluation platform and not just a standard chatbot

.

Login/Signup: You will use Supabase Auth for this, which is a production-ready, minimal-setup solution

. Completing this is the main deliverable of Phase 1

.

Upload Page: Created in Phase 2, this allows you to upload PDFs and store them in Supabase Storage

.

The Dashboard: This is the "Mission Control" of your app, where the real AI Engineering value is shown

.

#### 2. What the Dashboard Must Show

The dashboard is where you prove the system is reliable by displaying the following graphs and results:

Ragas Evaluation Metrics: You need to display scores for the four core industry-standard metrics: Faithfulness, Answer Relevance, Context Precision, and Context Recall

.

Experiment Leaderboard: A table or list that compares different configurations you have tested, such as different chunk sizes (200, 500, or 1000 characters) or prompt variations

. It should clearly highlight which version performed the best

.

Performance Comparison Charts:

Radar Charts (Spider Charts): As we discussed, these are the best for showing the balance between the four Ragas metrics

.

Line or Bar Charts: These should show performance gains—for example, showing how a "Chain-of-Thought" prompt improved your faithfulness score compared to a basic prompt

.

Observability Traces: While much of this happens in Langfuse, your dashboard should link to or summarize these "traces," which record the question, retrieved chunks, similarity scores, and the final evaluation for every interaction

.

### 3. Why this structure is required

Most basic portfolio projects only show a simple chat interface

. By including these specific pages and metrics, you demonstrate to recruiters that you understand RAG Systems, Evaluation Frameworks, and Performance Optimisation

. Using Next.js and Shadcn UI will give these results a professional, modern enterprise look that recruiters recognise

.

By the end of Phase 10, your dashboard will be a "Project Showcase" that features these live charts and a case study write-up, proving your AI system is accurate and reliable

.

How do I design the architecture diagram?

What should I include in my case study?

Can you help me summarize my final results?

I would like to implement these when designing frontend, all can be connected at later dates as development moves forward

That is a professional and highly effective way to approach the Phase 10: Portfolio Polish stage. Building the "shell" of your frontend early—using Next.js and Shadcn UI—gives you a clear roadmap of where each technical component will eventually live as your development progresses

.

As a Senior AI Engineer, I recommend building out this frontend skeleton with the following sections, which you can "hydrate" with real data from Supabase and OpenAI as you complete the underlying logic

:

### 1. The Homepage (The "Pitch")

This page should clearly state that this is NOT a chatbot project

. Use this space to explain that you are building a production-grade evaluation platform designed to solve the hallucination problem in AI

. Recruiters should immediately see that you are focusing on AI Engineering and Performance Optimisation

.

### 2. Login and Signup (The "Entry")

Since you have already set up Supabase Auth in Phase 1, these pages are your first step toward a "production-ready" feel

. Using a managed authentication system shows you prioritize security and industry-standard practices over building custom, less-secure alternatives

.

### 3. The Dashboard (The "Mission Control")

This is where the majority of your technical work from Phases 2 through 9 will eventually be displayed

. You can design the layout now with placeholders for the following:

Document Upload Area: A clean interface to handle PDF uploads, which will connect to your Supabase Storage and the Document Parser

.

QA Interface: The area where users ask questions and receive answers that are "grounded" in the retrieved context

.

The Metrics Hub (Ragas): Visual containers for your four core metrics: Faithfulness, Answer Relevance, Context Precision, and Context Recall

.

The Experiment Leaderboard: A table designed to compare different chunk sizes (200, 500, 1000) or prompt variations (like Chain-of-Thought)

. You can build the table now and simply leave it empty until you run your Experiment Engine

.

#### 4. Why This "UI-First" Strategy Works

Modern Enterprise Look: By using Shadcn UI and Tailwind CSS from the start, your project will maintain a professional appearance that recruiters recognize

.

Visualizing the Flow: Having the dashboard layout ready helps you visualize how data moves from Retrieval to AI Answer to Evaluation Engine

.

Debugging with Charts: Once you connect your Ragas metrics, seeing a Radar Chart or Line Graph update in real-time is the best way to debug if a specific experiment actually improved your system's performance

.

By designing these components now, you are essentially creating the "storyboard" for your final portfolio piece. As you move forward, you will simply be "plugging in" the live data streams from your backend into these professional visuals

.

Which Shadcn UI components should I use for the metrics?

How do I structure the final Case Study write-up?

Show me how to visualize the Ragas Radar Chart.

can you give me a prompt for my AI website builder for building frontend, also inform it what we are building and using supabase db, or is this too much info?

Providing this information to an AI website builder (like v0, Bolt.new, or Cursor) is not too much information; in fact, it is essential. Providing the specific technology stack and the unique "Evaluation Platform" goal ensures the AI generates a professional, enterprise-ready UI rather than a generic, hobbyist chatbot

.

Here is a comprehensive prompt you can use to generate the frontend skeleton for your AI Knowledge Base Evaluation Platform:

The Prompt for Your AI Website Builder

"I am building a production-grade AI Knowledge Base Evaluation Platform (not a basic chatbot) using Next.js, Tailwind CSS, and Shadcn UI. The backend is powered by Supabase for database, authentication, and storage.

Please build a professional, clean, and high-quality frontend skeleton with the following pages and components:

Landing Page (The Pitch): A hero section that highlights 'Accuracy-First AI' and 'Eliminating Hallucinations.' It should emphasize that this is an engineering platform for measuring and optimizing RAG systems

.

Auth Pages: Modern login and signup screens designed to work with Supabase Auth

.

Main Dashboard (The Mission Control): A layout with a sidebar containing the following sections:

Document Manager: A section for uploading PDFs and viewing metadata (linked to Supabase Storage)

.

QA & Retrieval Lab: An interface where users can ask questions and see the AI's answer side-by-side with the retrieved document chunks

.

Evaluation Hub: A section featuring placeholders for Ragas metrics (Faithfulness, Relevance, Precision, and Recall). Include a placeholder for a Radar Chart (Spider Chart) using a library like Recharts

.

Experiment Leaderboard: A professional Shadcn UI table designed to compare different RAG configurations (e.g., Chunk Size 200 vs 500 vs 1000)

.

Observability Summary: A simplified view showing status icons for 'Traces' that will eventually link to Langfuse

.

Design Requirements:

Use a modern 'Enterprise SaaS' aesthetic.

Ensure the layout is responsive and uses Shadcn UI components for buttons, cards, and tables

.

The UI should clearly differentiate between 'The AI Answer' and 'The Evaluation of the Answer' to demonstrate AI Engineering depth

."

Why This Info is Necessary

Supabase Mention: By mentioning Supabase, the AI builder will often suggest the correct data structures for user profiles and file uploads, saving you time later

.



The "NOT a Chatbot" Rule: This is the most important instruction. It forces the AI to prioritize the Evaluation Engine and Metrics Dashboard components, which are the "crown jewels" of your portfolio

.

Component Placeholders: Asking for placeholders for the Radar Chart and Leaderboard now allows you to focus on the complex logic of Phases 7 and 8 later without worrying about the layout

.

By using this detailed prompt, you are setting up the "storyboard" for your project, ensuring that when a recruiter visits your site, they see a professional engineering tool rather than a simple tutorial app

.

How should I structure the project's Case Study?

What are the most common RAG failures to monitor?

How can I showcase the Langfuse traces to recruiters?

OK, so once i have frontend skeleton completed, I move to step 2 in your plan

Correct, once your Phase 1 skeleton (including the working login system) is complete, we move directly to Phase 2: File Uploads

.

## 1. What We Are Building

In this phase, you are building the "front door" of your data pipeline

. You will create a dedicated page where you can upload PDF documents. These files will be stored in Supabase Storage, and their associated information (like file names and storage paths) will be recorded in your Supabase PostgreSQL database

.

## 2. Why It Matters

A production AI system is only as good as the data it can access

. Before we can chunk, embed, or evaluate anything, we must have a reliable way to ingest files

.

Recruiter Insight: Using a managed service like Supabase Storage shows you understand how to handle binary files (PDFs) separately from structured data (metadata), which is a standard production practice

.

## 3. Step-by-Step Implementation

To complete this phase, you need to perform the following tasks

:

Create the Upload Page: Build a new route in your Next.js app (e.g., /dashboard/upload) using Shadcn UI components like a File Input and an Upload Button

.

Initialize Supabase Storage: Go to your Supabase Dashboard and create a new "Storage Bucket" named documents. Set its privacy to "Public" for this stage so your application can easily access the files

.

Implement the Upload Logic: Write a function that uses the Supabase client to send the selected PDF to your documents bucket

.

Save Metadata: Once the upload to Storage is successful, immediately insert a new row into a documents table in your database containing the file\_name, file\_path, and the user\_id of the person who uploaded it

.

#### 4. Code Logic Explanation

Before you start writing the code, here is how the logic flows

:

Client-Side: The user selects a file. Your Next.js code catches that file object and sends it to Supabase Storage using `supabase.storage.from('documents').upload()`.

Server-Side/Database: You then take the unique path returned by the storage bucket and save it to your PostgreSQL documents table.

Separation of Concerns: We store the file itself in Storage because databases are not efficient at handling large PDF files. We store the link to the file in the database because it allows us to quickly search for and manage our documents

.

#### 5. Your Deliverable

A working upload page where you can select a PDF, click "Upload," and see that file appear in your Supabase Storage dashboard.

Please let me know once you have successfully uploaded your first document and verified it in your Supabase dashboard, and we will move to Phase 3: Document Processing

!

How do I setup a Supabase Storage bucket for PDFs?

What metadata should I store in the database for files?

Show me the code for the Next.js upload function.

shouldnt this step already be mentioned in website AI builder prompt. Create the Upload Page: Build a new route in your Next.js app (e.g., /dashboard/upload) using Shadcn UI components like a File Input and an Upload Button

You are absolutely correct. Including the Upload Page (Phase 2) in your initial frontend skeleton prompt is a smart move that will save you time later

.

While the "Senior AI Engineer" instructions for this project suggest moving through one phase at a time

, there is a logical reason to include the visual structure of the upload page in your initial AI builder prompt:

UI vs. Logic: Your AI website builder is great at creating the visual shell—the layout, the sidebar, and the file input button

. However, the actual engineering logic (connecting that button to Supabase Storage and your database) is what defines Phase 2

.

Completeness: By including the "Document Manager" or "Upload" section in your first prompt, you ensure that your Dashboard has a professional, finished look from day one. You can see in my previous prompt that I included a "Document Manager" section specifically for this reason.

The Roadmap:

Phase 1 (Skeleton): You are building the "look and feel" and the login system

.

Phase 2 (File Uploads): You will then write the specific code to make that "Upload" button actually work with Supabase Storage

.

Updated Advice for your AI Builder Prompt: If you haven't run the prompt yet, feel free to explicitly mention: "Include a dedicated '/dashboard/upload' route with a clean Shadcn UI file-drop zone and an 'Upload' button." This ensures the AI builder generates the specific page you'll need for Phase 2 immediately

.

In short: Use the AI builder to create the UI for the upload page now, and we will focus on the Backend Logic (the actual data flow) when we officially start Phase 2

.

How do I create the documents table in Supabase?

What fields should I include in the document metadata?

How do I handle multi-page PDFs during extraction?

add it to this prompt you gave me, then delete the other. "I am building a production-grade AI Knowledge Base Evaluation Platform (not a basic chatbot) using Next.js, Tailwind CSS, and Shadcn UI. The backend is powered by Supabase for database, authentication, and storage.

Please build a professional, clean, and high-quality frontend skeleton with the following pages and components:

Landing Page (The Pitch): A hero section that highlights 'Accuracy-First AI' and 'Eliminating Hallucinations.' It should emphasize that this is an engineering platform for measuring and optimizing RAG systems

.

Auth Pages: Modern login and signup screens designed to work with Supabase Auth

.

Main Dashboard (The Mission Control): A layout with a sidebar containing the following sections:

Document Manager: A section for uploading PDFs and viewing metadata (linked to Supabase Storage)

.

QA & Retrieval Lab: An interface where users can ask questions and see the AI's answer side-by-side with the retrieved document chunks

.

Evaluation Hub: A section featuring placeholders for Ragas metrics (Faithfulness, Relevance, Precision, and Recall). Include a placeholder for a Radar Chart (Spider Chart) using a library like Recharts

.

Experiment Leaderboard: A professional Shadcn UI table designed to compare different RAG configurations (e.g., Chunk Size 200 vs 500 vs 1000)

.

Observability Summary: A simplified view showing status icons for 'Traces' that will eventually link to Langfuse

.

Design Requirements:

Use a modern 'Enterprise SaaS' aesthetic.

Ensure the layout is responsive and uses Shadcn UI components for buttons, cards, and tables

.

The UI should clearly differentiate between 'The AI Answer' and 'The Evaluation of the Answer' to demonstrate AI Engineering depth

"

Why This Info is Necessary

Supabase Mention: By mentioning Supabase, the AI builder will often suggest the correct data structures for user profiles and file uploads, saving you time later

.

The "NOT a Chatbot" Rule: This is the most important instruction. It forces the AI to prioritize the Evaluation Engine and Metrics Dashboard components, which are the "crown jewels" of your portfolio

.

Component Placeholders: Asking for placeholders for the Radar Chart and Leaderboard now allows you to focus on the complex logic of Phases 7 and 8 later without worrying about the layout

.

By using this detailed prompt, you are setting up the "storyboard" for your project, ensuring that when a recruiter visits your site, they see a professional engineering tool rather than a simple tutorial app

.

This refined prompt combines the professional requirements of the Evaluation Platform with the specific instructions for the Phase 2 Upload Page visual shell. You can use this directly in your AI website builder (v0, Bolt.new, etc.) to generate the complete frontend skeleton.

The Final Comprehensive Frontend Prompt

"I am building a production-grade AI Knowledge Base Evaluation Platform (not a basic chatbot) using Next.js, Tailwind CSS, and Shadcn UI. The backend is powered by Supabase for database, authentication, and storage

.

Please build a professional, clean, and high-quality frontend skeleton with the following pages and components:

Landing Page (The Pitch): A hero section that highlights 'Accuracy-First AI' and 'Eliminating Hallucinations.' It should emphasize that this is an engineering platform for measuring and optimizing RAG systems

.

Auth Pages: Modern login and signup screens designed to work with Supabase Auth

.

Main Dashboard (The Mission Control): A responsive layout with a sidebar containing the following sections:

Document Manager & Upload Page: A dedicated route at /dashboard/upload. Include a clean Shadcn UI file-drop zone, a file input, and an 'Upload' button. It should also include a table to view document metadata (linked to Supabase Storage)

.

QA & Retrieval Lab: An interface where users can ask questions and see the AI's answer side-by-side with the specific retrieved document chunks to show grounding

.

Evaluation Hub: A section featuring placeholders for Ragas metrics (Faithfulness, Relevance, Precision, and Recall). Include a placeholder for a Radar Chart (Spider Chart) using the Recharts library

.

Experiment Leaderboard: A professional Shadcn UI table designed to compare different RAG configurations, such as 'Chunk Size: 200' vs 'Chunk Size: 500'

.

Observability Summary: A simplified view showing status icons for 'Traces' that will eventually link to Langfuse for production monitoring

.

Design Requirements:

Use a modern 'Enterprise SaaS' aesthetic.

Ensure the layout is responsive and uses Shadcn UI components for buttons, cards, and tables

.

The UI should clearly differentiate between 'The AI Answer' and 'The Evaluation of the Answer' to demonstrate deep AI engineering and performance optimisation

."

Why This Combined Prompt Works

Logical Workflow: By including the visual shell for the Upload Page now, you ensure your dashboard feels complete from day one. You'll be able to move from Phase 1 to Phase 2 by simply adding the "logic" to the buttons already designed by the AI

.

Recruiter Impact: The "NOT a Chatbot" instruction forces the AI to build a tool that looks like it belongs in a professional AI engineering team, highlighting evaluation and experimentation

.

Infrastructure Ready: Mentioning Supabase and Langfuse early ensures the AI builder creates a UI that can handle the specific data flows (like document metadata and trace logs) you will implement in the later phases

.

Once you have this skeleton live, your Phase 1 deliverable (a working login system) will be housed within a professional-looking environment, making it much easier to move into Phase 2: File Uploads.

How do I implement the upload logic in Phase 2?

What fields should my Supabase documents table include?

How can I test the Supabase storage bucket connection?

How do I implement the upload logic in Phase 2?

In Phase 2, we are building the "front door" of your data pipeline. This stage is critical because a production-grade system requires a reliable way to ingest and manage the documents that will eventually form your AI's knowledge base

.

### 1. What We Are Building

We are creating a secure file ingestion system that allows you to upload PDF documents to the cloud

. This involves two distinct storage actions:

Binary Storage: The actual PDF file is stored in Supabase Storage

.

Metadata Storage: Details about the file (name, storage path, and which user uploaded it) are stored in your Supabase PostgreSQL database

.

### 2. Why It Matters

In production AI engineering, we never store large files like PDFs directly inside a database because it is inefficient and expensive

. Instead, we use Supabase Storage for the heavy files and the PostgreSQL database for the "index" or metadata

. This separation of concerns ensures your application remains fast and scalable as you add more documents

.

### 3. Step-by-Step Implementation Instructions

#### Step 1: Configure Supabase Storage

Log into your Supabase Dashboard.

Navigate to the Storage tab and create a new bucket named documents

.

Set the bucket to "Public" for this phase to simplify access while we build the core logic

.  
Step 2: Create the Database Table In your Supabase SQL editor, create a documents table to store the metadata

:

```
create table documents (  
  id uuid primary key default uuid_generate_v4(),  
  name text not null,  
  storage_path text not null,  
  user_id uuid references auth.users(id),  
  created_at timestamp with time zone default now()  
);
```

Step 3: Build the Upload Route Create a new page in your Next.js project (e.g., /dashboard/upload)

. Use Shadcn UI components to build a file input and an "Upload" button for a professional enterprise look

.

Step 4: Implement the Client-Side Logic You will need to write a function that handles the file selection and communicates with Supabase. The logic follows a two-step process:

Upload to Storage: Send the file to the documents bucket

.

Record Metadata: If the upload is successful, take the returned file path and insert a new row into your documents database table

.

#### 4. Code Logic Explanation

Before you write the code, understand these two key Supabase Client methods you will use:

`supabase.storage.from('documents').upload(path, file)`: This method handles the binary transfer of your PDF from the user's browser to Supabase's cloud storage

. The path should be unique (often a combination of the user ID and a timestamp) to prevent files from overwriting each other.

`supabase.from('documents').insert([...])`: This is a standard database insert

. It doesn't save the file content, but rather the "address" of where the file lives in storage. This allows the Document Parser in Phase 3 to find the file and begin chunking the text

.

Deliverable: You should be able to upload a PDF via your Next.js frontend and see the file appear in your Supabase Storage dashboard and a new record appear in your PostgreSQL table



Once you have verified that your PDF uploads are working correctly, we can proceed to Phase 3: Document Processing, where we will begin extracting text and splitting it into searchable chunks

What columns should the Supabase documents table include?

How do I handle multiple file uploads simultaneously?

How should I name files in Supabase Storage?

What columns should the Supabase documents table include?

Based on the project roadmap and our previous discussion regarding Phase 2: File Uploads, your documents table in Supabase should be designed to store critical metadata that links your physical files in storage to your structured data in the database

To maintain a production-grade architecture, the table should include the following columns:

**id:** A unique identifier for each document (typically a uuid). This is essential for referencing the document in later phases, such as when storing searchable chunks in Phase 3

**name:** The original name of the PDF file. This is used for the Document Manager UI in your frontend so you can easily identify what has been uploaded.

**storage\_path:** The exact location or "address" of the file within Supabase Storage

. This is critical for the Document Parser to find the file and extract text during Phase 3

**user\_id:** A reference to the user who uploaded the file, linked to Supabase Auth

. This ensures that users only see and interact with their own documents, which is a requirement for a secure, multi-user platform.

**created\_at:** A timestamp indicating when the file was uploaded. This is useful for sorting files in your dashboard and tracking performance trends over time in your Metrics Dashboard

## Implementation SQL

You can create this table in your Supabase SQL editor using the following command:

```
create table documents (  
  id uuid primary key default uuid_generate_v4(),  
  name text not null,
```

```
storage_path text not null,  
user_id uuid references auth.users(id),  
created_at timestamp with time zone default now()  
);
```

Why these columns matter

Storing the metadata (like the storage\_path) instead of the actual file content in your database is a standard production practice because it keeps your database fast and efficient

. This setup allows the Retriever in Phase 5 to quickly identify which documents are being used to generate "grounded" answers for your AI system

.

How should I secure the documents table using RLS?

What is the best way to handle large file uploads?

I've created the table, what's next for Phase 2?

How should I secure the documents table using RLS?

To secure your documents table using Row Level Security (RLS) in Supabase, you must ensure that users can only interact with the files they have uploaded themselves. This is a critical step for a production-grade architecture, as it prevents data leaks and ensures privacy in a multi-user environment

.

As we are currently in Phase 2: File Uploads, setting up RLS now ensures that your document metadata is protected before you begin processing it in Phase 3

.

## 1. What We Are Building

We are creating "gatekeeper" rules directly within your Supabase PostgreSQL database

. These rules will automatically check the identity of the user making a request and only allow them to see, add, or delete rows where the user\_id matches their own unique authenticated ID.

## 2. Why It Matters

In a professional AI platform, security is not an afterthought. RLS ensures that:

Privacy: One user cannot see another user's private documents or evaluation results

.

Data Integrity: Users can only modify or delete their own data.

Production Readiness: Recruiters look for RLS because it proves you know how to build secure, scalable applications using industry-standard tools

### 3. Step-by-Step Implementation Instructions

You can run these commands in the Supabase SQL Editor:

Step 1: Enable RLS on the Table By default, RLS is disabled, meaning anyone with the API key could potentially see all data. This command "locks the door":

```
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;
```

Step 2: Allow Users to View Their Own Documents This policy allows a user to SELECT (read) rows only if their auth.uid() matches the user\_id column in the table:

```
CREATE POLICY "Users can view their own documents"
```

```
ON documents FOR SELECT
```

```
USING (auth.uid() = user_id);
```

Step 3: Allow Users to Upload Documents This policy ensures that when a user INSERTs a new row, the user\_id they provide must be their own:

```
CREATE POLICY "Users can insert their own documents"
```

```
ON documents FOR INSERT
```

```
WITH CHECK (auth.uid() = user_id);
```

Step 4: Allow Users to Delete Their Own Documents This is important for the "Document Manager" part of your dashboard so users can clean up their files:

```
CREATE POLICY "Users can delete their own documents"
```

```
ON documents FOR DELETE
```

```
USING (auth.uid() = user_id);
```

### 4. Code Logic Explanation

auth.uid(): This is a special function provided by Supabase Auth

. It automatically extracts the ID of the logged-in user from their secure session token.

USING vs. WITH CHECK:

USING is for existing data (can I see or delete this row?).

WITH CHECK is for new data (am I allowed to add this row with these specific values?).

Security at the Source: Because these rules live in the database, even if someone tries to hack your frontend code, they cannot bypass these rules to see other users' PDFs

### 5. Final Checklist for Phase 2

Before moving to Phase 3: Document Processing, ensure you have:

Frontend Skeleton: Created the landing page, login, and dashboard layout

.  
Upload Logic: Successfully uploaded a PDF to Supabase Storage

.  
Metadata Storage: Saved the file name and path in the documents table

.  
RLS Secured: Applied the policies above to keep the data private.

Once you have verified that you can only see your own uploads in the dashboard, let me know, and we will begin Phase 3: Document Processing to turn those PDFs into searchable text chunks

!

How do I extract text and create chunks in Phase 3?

Which library should I use for PDF text extraction?

What is the best chunking strategy for LlamaIndex?

What is the best way to handle large file uploads?

According to the sources and our project roadmap, the best way to handle large file uploads in a production-grade AI system is through a separation of concerns between binary storage and metadata tracking

.  
In this architecture, you should follow these three core principles:

### 1. Use Managed Cloud Storage for Binary Files

Instead of storing large PDFs directly in your database—which is inefficient and expensive—you should use Supabase Storage

. This service is purpose-built to handle large file transfers and binary data, ensuring your application remains fast and scalable

### 2. Store Only Metadata in the Database

Once the file is successfully uploaded to cloud storage, you should record only the metadata in your Supabase PostgreSQL database

. This includes:

File Name: For display in your dashboard

.  
Storage Path: The specific "address" used to retrieve the file from storage later

User ID: To ensure the document is linked to the correct account for security purposes.

### 3. Implement an "Ingestion Pipeline"

The sources outline a specific flow that begins with the upload and leads into the rest of the AI engineering process

:

User Uploads PDF: Handled by a professional frontend (using Next.js and Shadcn UI)

.

Storage: The file is saved to a dedicated bucket in Supabase Storage

.

Database Entry: Metadata is saved to the PostgreSQL documents table

.

Downstream Processing: This storage path is then passed to the Document Parser and Chunking steps in Phase 3 to begin converting the file into searchable knowledge

.

By using this method, you demonstrate Production AI Development and AI Engineering skills to recruiters by showing you understand how to manage large-scale data ingestion without compromising database performance

.

How do I test the RLS policies in Supabase?

What is the next step for Phase 3: Processing?

How should I format the metadata for the database?

What is the best chunking strategy for LlamaIndex?

In the context of this project, the "best" chunking strategy is not a fixed number but rather the one that you identify through data-driven experimentation. According to the sources, there is no universal best size; instead, you are building an Experiment Engine specifically to find the optimal configuration for your unique documents

.

To determine the best strategy for your LlamaIndex pipeline, you should follow these steps outlined in the development roadmap:

#### 1. Experiment with Specific Sizes

Phase 8 of the project explicitly tasks you with testing three different chunk sizes to see how they impact your results:

200 characters (Smaller, more granular chunks)

.  
500 characters (Medium-sized balance)

.  
1000 characters (Larger chunks with more context)

.  
**2. Measure Performance with Ragas**

The "best" strategy is the one that achieves the highest scores across the four core Ragas metrics

:

Context Precision: Does the smaller chunk size help the retriever find exactly what is needed?

Context Recall: Does a larger chunk size (like 1000) ensure no necessary information is left out?

Faithfulness: Does the chunk size provide enough context for the AI to answer without hallucinating?

**3. Use a Scientific Approach**

The goal of this platform is to move away from "guessing" what works. By testing these variations and recording them in your Experiment Leaderboard, you can visually prove to a recruiter which chunk size is superior for your specific dataset

.  
As your Senior AI Engineer, I recommend starting with 500 characters as your baseline, then testing 200 and 1000 to see if your Context Precision or Recall improves. This methodology demonstrates the Performance Optimisation and Experiment Design skills that define a production-grade AI engineer

.  
How do I extract text from a PDF in Phase 3?

What is the difference between chunking and embedding?

How do I store the processed chunks in Supabase?

How do I test the RLS policies in Supabase?

Testing your Row Level Security (RLS) policies is a critical step in Phase 2: File Uploads to ensure your application meets the "production-grade" standards required for an AI Knowledge Base Evaluation Platform

. While the provided sources focus on the architectural goals and roadmap, I can provide the standard industry methods for verifying these security policies to ensure your metadata remains private and secure

.  
**1. The Multi-User Session Test (Most Realistic)**

Since you completed Phase 1: Project Setup and have a working login system, the best way to test RLS is to simulate real users

.

Step 1: Open your application in a normal browser window and sign in as User A. Upload a document.

Step 2: Open a private/incognito window and sign in as User B.

Step 3: Navigate to your dashboard. If your RLS policies are correct, User B should not see User A's document in the list, even though they are both looking at the same documents table.

Why it works: This verifies that the `auth.uid() = user_id` policy is correctly identifying the active session and filtering the data at the database level

.

## 2. Testing via the Supabase SQL Editor (Fastest)

You can simulate different users directly inside the Supabase Dashboard to test your policies without logging in and out of the frontend. Note that this information on SQL simulation is not explicitly in your sources, but it is the standard way to debug Supabase RLS.

Simulate a User Session: You can run a command to "act" as a specific user.

Step 1: Find a `user_id` from your `auth.users` table.

Step 2: Run the following in the SQL Editor:

Result: You should only see rows that belong to that specific `user_id`. If you change the ID to a different user, the results should change accordingly.

## 3. The "Service Role" Comparison

To confirm RLS is actually "on," you can compare the results from your API (which respects RLS) against the Service Role Key (which bypasses RLS).

The Test: If you query the table using your anon key or via the dashboard UI (while RLS is enabled), you should see limited results.

The Verification: If you use the `service_role` key (found in your Supabase settings), you will see all documents. This confirms that the database is correctly enforcing restrictions for standard users but allowing "admin" access where necessary.

## Why Testing Matters Now

In this project, your goal is to move beyond a simple "document upload" to a Production AI Development workflow

. By verifying RLS now, you guarantee that when you reach Phase 7 (Evaluation) and Phase 9 (Observability), your traces and metrics are strictly isolated to the correct user, preventing one user from seeing another's "Chain-of-Thought" prompts or Ragas scores

.

Once you have verified that User A cannot see User B's metadata, you have officially secured your data pipeline and are ready for Phase 3: Document Processing.

I have verified my uploads, let's start Phase 3.

Which library should I use to extract text from PDFs?

How do I split extracted text into searchable chunks?

now how do i split the steps and prompts you have ,given me from answers to my questions.. so i can keep these 2 informations...

To help you keep your Project Execution (the "how-to" steps and prompts) separate from your Technical Knowledge Base (the "why" and conceptual answers), I will create two distinct reports for you. This will allow you to have a clean manual for building the platform and a separate reference guide for the technical concepts.

### 1. The Engineering Playbook (Steps & Prompts)

This document will focus exclusively on the Development Roadmap and the specific Prompts you need to give to your AI tools. It includes:

The 10-Phase Roadmap: Every task from Phase 1 (Setup) to Phase 10 (Portfolio Polish)

.

The Master Frontend Prompt: The comprehensive prompt we designed for your AI website builder to create your professional dashboard and upload pages.

Phase-Specific Deliverables: A checklist of what you should have at the end of each step

.

### 2. The Technical Knowledge Base (Concepts & Q&A)

This document will capture the Senior AI Engineer insights and answers to the deeper technical questions we've discussed. It includes:

Core Concepts: Detailed explanations of RAG Systems, Grounded AI, and Hallucination Mitigation

.

Evaluation & Metrics: Deep dives into Ragas (Faithfulness, Relevance, Precision, Recall) and why they are industry standards

.

Advanced Techniques: Explanations of Chain-of-Thought (CoT) prompting, Chunking Strategies, and Observability with Langfuse

.

Security & Infrastructure: Best practices for Supabase RLS and handling large file uploads

.



I am initiating the creation of these two tailored reports now to help you organize this information.

I have initiated the creation of two distinct reports for you to split your information effectively.

The first report, the Development Playbook, contains the procedural steps for all 10 phases and the master prompts for your AI builder

. The second report, the Technical Knowledge Base, serves as your conceptual reference for things like Ragas metrics, observability, and security

.

These reports are being created now and will be visible in your studio tab shortly. They should help you maintain a clear distinction between the "how" and the "why" of your project. Is there anything specific from our recent discussion on Phase 2: File Uploads or RLS that you want to make sure is highlighted in these guides?

Highlight the security logic for the RLS guide

How do I extract text from PDFs in Phase 3?

Which libraries should I use for document chunking?

can i also have an infographic

I have initiated the creation of an infographic for you.

This visual will map out the complete 10-phase development roadmap and the technical architecture of your platform, from the initial document upload through to the final evaluation engine and metrics dashboard

.

It is being generated asynchronously and will appear in your Studio tab shortly. This will serve as a great visual reference for your portfolio or to help you track your progress as we move through the remaining phases!

How can I represent this infographic in my portfolio?

What are the specific extraction tasks in Phase 3?

How do I extract text from a PDF with LlamaIndex?

i think i will need to copy what you gave me im sure theres information missed in the docs

You are right to be cautious. While the original source document provides a high-level roadmap, our conversation has added significant "production-grade" implementation details, specific code logic, and the master prompts required to actually build the platform

.

To ensure you don't lose the specific instructions that were generated during our chat, here are the most critical "chat-exclusive" details you should save alongside the reports I've created:

## 1. The Master Frontend Prompt (for v0 / Bolt.new)

This is the single most important piece of "extra" information we developed to ensure your UI reflects an AI Engineering platform rather than a simple chatbot

.

Tip: Copy the comprehensive prompt we finalized in our last few messages to give to your AI builder.

## 2. The Row Level Security (RLS) SQL

While the source mentions using Supabase, it doesn't provide the specific security policies we discussed to keep user data private

. You should save these SQL commands to run in your Supabase editor:

Enable RLS: ALTER TABLE documents ENABLE ROW LEVEL SECURITY;

Select Policy: USING (auth.uid() = user\_id);

Insert Policy: WITH CHECK (auth.uid() = user\_id);

## 3. Prompt Variation Strategy (Chain-of-Thought)

In our discussion about Phase 8 (Experiment Engine), we added the specific "Chain-of-Thought" strategy that wasn't in the original document

.

The Concept: Forcing the AI to "think step-by-step" to improve Faithfulness and Answer Relevance scores

.

The Implementation: Using two distinct templates (Simple vs. Expert Analyst) to prove you can scientifically "tune" AI behavior

.

## 4. Observability Logic (Langfuse)

The source mentions Langfuse generally, but our chat detailed the Trace vs. Span hierarchy you need to implement in Phase 9

:

Parent Trace: The full interaction.

Retrieval Span: Logging the specific chunks from Supabase.

Generation Span: Logging the OpenAI prompt and response.

## 5. Recommended Visual Charts

For your Phase 10 Portfolio Polish, we identified the specific charts that demonstrate "Performance Gains" to recruiters

:

Radar Chart: For the four Ragas metrics (Faithfulness, Relevance, Precision, Recall).

Line/Bar Charts: For comparing experiment winners (e.g., Chunk Size 500 vs. 1000).

How to save this: I have already incorporated these specific prompts and technical details into the "Development Playbook" and "Technical Reference" reports currently in your studio tab. However, I highly recommend copying the "Master Frontend Prompt" directly from our chat history, as it is a ready-to-use tool for your next step in building the UI.

Would you like me to create one more artifact that is just a "Code & Prompt Snippet Library" containing only the copy-pasteable code and prompts we've created?

Yes, please create the Code & Prompt Snippet Library.